

Master of Engineering Thesis

submitted in fulfillment of the requirements for the degree of Master of Engineering

in the study programme

Artificial Intelligence for Smart Sensors / Actuators

Development of a SLAM and Autonomous Navigation System for a Self-Driving Car

submitted by

Amit Sajeev

Matriculation Number: 22306894

Supervisor: Prof. Dr. Dmitrii Dobriborsci

Deggendorf Institute of Technology

14.09.2026

Statement of Authorship

I hereby declare that I have authored this thesis independently, that I have not used any sources or resources other than those cited, and that all passages taken literally or in substance from published or unpublished works are identified as such. I further declare that this thesis has not previously been submitted, in this or a similar form, to any other examination board.

Place, Date

Amit Sajeev

Abstract

Autonomous mobile robots are typically developed in simulation before deployment on physical hardware, yet the transfer to a real platform routinely exposes gaps that simulated testing alone does not predict. This thesis documents the migration of a ROS 2 Nav2 navigation stack for the Quanser QCar, an Ackermann-steered vehicle, from Gazebo simulation onto physical hardware. An incompatibility between the robot’s onboard ROS 2 Dashing distribution and the ROS 2 Humble development host motivated a custom TCP/JSON bridge. The thesis presents the bridge’s design, the calibration of an open-loop throttle model against real actuator behaviour, and the diagnosis of a hardware buffering defect misattributed at first to drivetrain coasting. The resulting system is validated through closed-loop Nav2 goal-reaching tests on the vehicle, and the observed sim-to-real discrepancies are quantified.

Key words: ROS 2, Nav2, Ackermann steering, sim-to-real transfer, Gazebo, Model Predictive Path Integral control, mobile robot navigation, Quanser QCar

Contents

Abstract	ii
Index of Abbreviations	viii
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	1
1.3 State of the Art	2
1.4 Methodological Approach	2
1.5 Thesis Structure	3
2 Groundwork and Fundamental Principles	4
2.1 The ROS 2 Communication Model	4
2.1.1 DDS Discovery: Why an Incompatibility Manifests as Silence	5
2.1.2 The Action Interface and Goal Lifecycle	6
2.2 Ackermann Steering Kinematics	6
2.2.1 The True Two-Wheel Ackermann Angle	7
2.3 Simulation with Gazebo	8
2.3.1 The Plugin’s Control Loop: PID and the Physics Step	9
2.3.2 Contact Friction: the Coulomb Model and Friction Cone	9
2.4 Simultaneous Localization and Mapping: Cartographer	10
2.5 The Nav2 Navigation Stack	10
2.5.1 MPPI in Detail: Sampling, Scoring, and the Softmax Update	11
2.5.2 AMCL in Detail: The Particle Filter and the Recovery Mechanism	12
2.6 The Quanser QCar Platform	13
2.7 The Sim-to-Real Gap	14
3 Simulation Stack Development	15
3.1 Baseline: Manual Bicycle-Model Inversion	15
3.2 First Migration: <code>ros2_control</code> ’s <code>AckermannSteeringController</code>	15
3.2.1 A Wheel-Axis Bug Exposed by the Migration	16
3.2.2 A URDF Parser Defect in <code>gazebo_ros2_control</code>	17
3.3 Porting the Navigation Stack to ROS 2 Humble / Gazebo Classic	17
3.4 Second Migration: The Direct Drive Plugin	18
3.4.1 A Quiet Regression: Odometry Reverts to Ground Truth	19
3.5 The “Robot Not Moving Forward” Bug Pattern	19
3.5.1 Diagnostic Method	22
3.6 A Behaviour-Tree Fix for Ackermann Recovery	22
3.7 Summary of the Simulation-Stack Architecture	24

4	Real-Hardware Bridge Architecture	25
4.1	The ROS 2 Dashing / Humble Incompatibility	25
4.2	Bridge Architecture Overview	25
4.3	LiDAR Angle-Convention Correction	27
4.4	Wheel Odometry over the Bridge	28
4.5	Steering-Trim Calibration	30
4.5.1	Steering Gain: A Second, Independent Nonlinearity	30
4.5.2	A Trim-Placement Bug and the “Apply at the Actuator Boundary” Principle	32
4.6	Clock Synchronisation and the Receipt-vs-Capture Timestamp Bug	33
4.7	Safe Shutdown Procedure	35
4.8	Summary	35
5	System Identification and Calibration	36
5.1	Problem Statement	36
5.2	Ruling Out Simpler Explanations	36
5.3	Active Braking: A Control-Design Attempt and a Safety Incident	37
5.4	Sim-versus-Real Comparison	38
5.5	Throttle Calibration	39
5.5.1	The Uncalibrated Gain Hypothesis	39
5.5.2	First Calibration Attempt: A Deadband Discovered, Then Reverted . . .	39
5.5.3	Final Calibration: An Empirical Duty Sweep	40
5.5.4	A Stray-Process False Positive	42
5.6	The Hardware Abstraction-Layer Buffering Defect	42
5.6.1	An Initial, Incomplete Diagnosis	42
5.6.2	Disproving Two Hypotheses, and the Argument That Found the Real Cause	43
5.6.3	Root Cause: a Ring-Buffer I/O Mode	43
5.6.4	Validation	45
5.7	Summary	46
6	Navigation Integration and Validation	47
6.1	Porting the Nav2 Configuration to Hardware	47
6.2	Custom MPPI Critic: <code>EarlyCommitCritic</code>	47
6.3	Phased Hardware Integration	49
6.4	A K-Turn Freeze at Reversal Cusps: Diagnosis and a Custom Smoother	50
6.4.1	Reproduction	50
6.4.2	Diagnosis: The Planner Is Correct, the Controller Is Not	50
6.4.3	Three Plausible Fixes, All Reverted	51
6.4.4	A Reverse-Only, Gradually Blended Smoother	52
6.4.5	“Extending the Corner”: A Further, Larger Improvement	53
6.4.6	Real-Hardware Validation	54
6.4.7	Continued Iteration: A Re-Extension Bug and a Curvature-Continuing Refinement	54
6.4.8	A Related Planner-Side Finding: Near-Wall Goals and a Tolerance Trade-off	56

6.4.9	Two Follow-On Attempts, Reverted	57
6.4.10	A Related Operational Caution: Goal Preemption	57
6.5	Diagnosing an Apparent Display-Localisation Artefact	58
6.5.1	Ruling Out Simpler Causes	58
6.5.2	Instrumentation: <code>monitor_goal_timing.py</code>	58
6.5.3	Measured Result	60
6.5.4	Interpretation, and a Necessary Caveat	60
6.6	Final Validation	61
6.7	Summary	61
7	Results	62
7.1	Actuator Calibration Accuracy	62
7.2	Stop-Distance and Latency	62
7.3	Sim-versus-Real Overshoot	63
7.4	Steering Response Accuracy	63
7.5	Navigation Robustness in Tight Spaces	63
7.6	Navigation Goal-Reaching	64
7.7	Applicability of the Results	64
7.7.1	Beyond This Specific QCar Unit	64
7.7.2	Beyond the QCar Platform	64
7.7.3	Beyond Navigation	65
8	Summary and Prospects	66
8.1	Summary	66
8.1.1	Critical Reflection	67
8.2	Prospects	67
	Bibliography	69
A	Appendix	71
A.1	QCar Hardware Safety Limits	71
A.2	Diagnostic and Calibration Tooling Reference	71
A.3	Real-Hardware Bring-Up Command Reference	72
A.3.1	Safe Shutdown Sequence	72
A.4	Source Code and Data Availability	72

List of Figures

List of Tables

2.1	QoS compatibility rule for the two policies relevant to this thesis (strongest requestable value shown first).	5
2.2	QCar Ackermann geometry parameters used throughout this thesis.	8
3.1	Gazebo Sim (Jazzy) to Gazebo Classic (Humble) translation table.	18
3.2	The recurring “not moving forward” bug pattern, as found in both affected packages.	20
4.1	Illustrative raw-to-corrected LiDAR angle mapping (Equation 4.1).	28
4.2	Commanded steering angle versus implied real wheel angle (<code>STEERING_TRIM=0</code>).	32
4.3	Diagnostic measurements underlying the clock-offset fix.	35
5.1	Fixed-duty sweep results (QCar on the ground, real load).	40
5.2	Throttle calibration validation (Equation 5.2).	41
5.3	Effect of the <code>readMode=0</code> fix on stiction delay, extra distance, and stop latency.	45
6.1	K-turn manoeuvre duration and recovery count, corridor reproduction goal (simulation).	53
6.2	Timing trace of one Nav2 goal (<code>monitor_goal_timing.py</code>).	60
7.1	Commanded-versus-real cruise speed, before and after calibration.	62
7.2	Stop-latency and extra-distance results, before and after the <code>readMode=0</code> fix.	62
7.3	Sim-versus-real stop-distance overshoot (measured before the fixes of Ch. 5).	63
7.4	Real wheel deflection versus kinematically commanded deflection, before and after <code>STEERING_GAIN</code> correction.	63
7.5	K-turn manoeuvre duration and recovery count, before and after the cusp-straightening smoother (simulation, three trials each).	64
A.1	QCar documented hardware safety limits.	71
A.2	Diagnostic and calibration scripts referenced in this thesis.	71

Index of Abbreviations

AMCL	Adaptive Monte Carlo Localization
API	Application Programming Interface
BT	Behavior Tree
CPU	Central Processing Unit
DDS	Data Distribution Service
DIT	Deggendorf Institute of Technology
GUI	Graphical User Interface
HAL	Hardware Abstraction Layer
IMU	Inertial Measurement Unit
IP	Internet Protocol
JSON	JavaScript Object Notation
LiDAR	Light Detection and Ranging
MPPI	Model Predictive Path Integral (control)
NTP	Network Time Protocol
PAL	Product Application Layer (Quanser)
PID	Proportional-Integral-Derivative (controller)
PWM	Pulse-Width Modulation
QoS	Quality of Service
RMW	ROS Middleware (interface)
ROS	Robot Operating System
RViz	ROS Visualization (tool)
SDF	Simulation Description Format
SLAM	Simultaneous Localization and Mapping
SSH	Secure Shell
TCP	Transmission Control Protocol
TF	Transform (ROS coordinate-frame library)
UDP	User Datagram Protocol
URDF	Unified Robot Description Format
XML	Extensible Markup Language
YAML	YAML Ain't Markup Language

1 Introduction

1.1 Motivation

Autonomous mobile robots are, almost without exception, developed in simulation first. A simulator offers unlimited, repeatable, and safe experimentation: a planner can be tuned against a thousand simulated collisions before it is ever allowed to steer a physical vehicle, and a software defect that would damage hardware in the real world merely crashes a process in simulation. This development pattern is so deeply embedded in contemporary robotics practice that entire open-source ecosystems – the Robot Operating System (ROS) and its successor ROS 2 [9, 4], the Gazebo simulator [3], and the Nav2 navigation stack [5] used throughout this thesis – are built around the assumption that a robot’s software stack should run, largely unmodified, against either a simulated or a physical robot.

That assumption is also where sim-to-real transfer earns its reputation as one of the harder open problems in applied robotics [14]. A simulator models the physics, sensors, and actuators its authors chose to model, at the fidelity they chose to model them; a physical robot is subject to every effect nobody chose to model at all – static friction, actuator lag, battery voltage sag, unsynchronised clocks between distributed compute nodes, and firmware-level buffering nobody documented. The gap between the two is not a single bug to be fixed once, but a class of problems that resurfaces every time a new subsystem crosses from simulation onto hardware.

This thesis is a case study of that gap, built around a single physical platform: the Quanser QCar, a 1/10th-scale, real-wheel-drive, Ackermann-steered research vehicle equipped with a 2D LiDAR, an inertial measurement unit (IMU), and an onboard NVIDIA Jetson TX2 compute module. The QCar’s navigation stack – SLAM-based mapping, Adaptive Monte Carlo Localization (AMCL), and Model Predictive Path Integral (MPPI) trajectory control [13, 12] – was first developed and validated end-to-end in a Gazebo simulation, then migrated onto the physical vehicle. The migration surfaced a sequence of concrete, individually diagnosable problems: a protocol-level incompatibility that ruled out native ROS 2 communication with the robot’s onboard compute, an uncalibrated and non-linear mapping from commanded velocity to real actuator duty cycle, and – ultimately the most consequential finding of this project – a hardware abstraction-layer defect that had, for several weeks, been mistaken for a fundamental physical property of the vehicle’s drivetrain.

1.2 Problem Statement

At the outset of this project, the QCar’s navigation stack existed only in simulation: a Gazebo model of the vehicle, driven by the same Nav2 stack intended for the physical robot, could map an environment, localise itself within it, and reach goal poses reliably. The *actual state* was therefore “a navigation stack validated in simulation, with no confirmed path to the physical robot”. The *target state* for this thesis was a navigation stack that reaches goal poses reliably *on the physical QCar*, with any sim-to-real discrepancy identified, explained, and – where practical – corrected or compensated for rather than left as an unexplained residual error.

Reaching that target state required answering four sub-questions, each of which forms one methodology chapter of this thesis:

1. How should the simulation-side navigation stack itself be structured so that migrating it to hardware later requires changing as little of its Nav2/SLAM configuration as possible? (Chapter 3)
2. Given that the robot’s onboard compute cannot run the same ROS 2 distribution as the navigation stack, how should the two be connected so that the rest of the stack does not need to know the difference? (Chapter 4)
3. How should the mapping from a ROS 2 velocity command to a physical actuator signal be identified and calibrated, given that the vehicle exhibits both an unknown gain and a hard static-friction non-linearity? (Chapter 5)
4. Once bridged and calibrated, does the navigation stack actually reach goals reliably on hardware, and what residual sim-to-real gap remains? (Chapters 6 and 7)

1.3 State of the Art

ROS 2 was designed specifically to address shortcomings of the original ROS that made it unsuitable for production and safety-relevant use, chief among them a lack of real-time support and a single-point-of-failure master node; it replaces ROS’s custom TCP transport with the Data Distribution Service (DDS) publish-subscribe middleware standard [7, 4, 6]. Nav2 is the de facto standard ROS 2 navigation stack, providing pluggable costmaps, global planners, and local trajectory controllers behind a behaviour-tree-orchestrated action interface [5]. MPPI control, the local trajectory controller used in this project, samples large numbers of candidate control sequences and weights them by a differentiable cost function evaluated over a rollout horizon, avoiding the need for a convex or otherwise analytically tractable vehicle model [13, 12] – a property well suited to an Ackermann-steered vehicle whose real dynamics (addressed in Chapter 5) turned out to be considerably less clean than the textbook bicycle model in Chapter 2.

Sim-to-real transfer itself is an active research area, most visibly in reinforcement-learning contexts where a policy trained purely in simulation is deployed with no further real-world tuning [14]. This thesis does not train a learning-based policy and therefore sits outside that literature’s core concern (closing the "reality gap" for a black-box learned controller); its navigation stack – SLAM, AMCL, and MPPI – is classically engineered and identically configured in both simulation and reality. What this thesis instead documents is a narrower but still underserved question in the same space: for a classically engineered stack with an explicit vehicle/sensor/actuator model, what concrete, individually diagnosable failure modes appear when that same stack meets real hardware, and how are they found?

1.4 Methodological Approach

The work reported in this thesis followed an empirical, iterative methodology rather than a purely analytical one: each sim-to-real discrepancy was investigated by direct measurement

on the physical robot (odometry logs, encoder-measured velocity, timestamped Nav2 action status, and, in one case, reading the vendor’s hardware abstraction layer source code directly) rather than inferred solely from documentation or first-principles modelling. This approach was not a stylistic preference; Chapter 5 documents a case in which two independently plausible physical explanations for an observed 2-second control delay – drivetrain coasting and a software command-delivery lag – were both consistent with the available second-hand evidence, and only a targeted hardware measurement, prompted by a direct behavioural observation that contradicted both prevailing theories, resolved which was correct (Section 5.6).

One control-design attempt in this project was tested directly on the physical vehicle without first being traced through by hand or validated in simulation, and it produced a genuine, unplanned hardware oscillation (Section 5.3). That incident is reported in full, including why it happened and why it was reverted rather than iterated on further, because it is as methodologically relevant to a sim-to-real case study as any of the project’s successful fixes: it demonstrates concretely why validating control logic before it reaches physical actuators is not merely good practice but a documented, tested necessity for this class of system.

1.5 Thesis Structure

Chapter 2 introduces the technical foundations this thesis builds on: the ROS 2/DDS communication model, Ackermann steering kinematics, the Gazebo simulator and its ROS 2 integration, SLAM via Cartographer [2], and the Nav2 stack’s AMCL localisation and MPPI control components. Chapter 3 describes the simulation-side development of the navigation stack, including two rounds of a systematic vehicle dynamics bug that recurred across sibling packages, and the discovery and fix of a URDF parser defect in the underlying middleware. Chapter 4 presents the design of the TCP/JSON bridge that connects the ROS 2-based navigation stack to the robot’s incompatible onboard ROS 2 distribution, including the odometry, LiDAR angle-convention, and steering-trim fixes required to make the bridged sensor and actuator data trustworthy. Chapter 5 covers the identification of the vehicle’s throttle response, including the static-friction deadband and the hardware abstraction-layer buffering defect that had been mistaken for physical coasting. Chapter 6 describes integrating and tuning the full Nav2 stack – including a custom MPPI critic plugin and AMCL parameter changes – against the physical robot, culminating in closed-loop goal-reaching tests. Chapter 7 presents and quantifies the resulting sim-to-real gap across the project’s diagnostic measurements, and Chapter 8 answers the research questions posed above and reflects on what generalises beyond this specific platform.

2 Groundwork and Fundamental Principles

This chapter introduces the concepts and software components that the remainder of the thesis builds on: the ROS 2 communication model (Section 2.1), Ackermann steering kinematics (Section 2.2), the Gazebo simulator (Section 2.3), SLAM via Cartographer (Section 2.4), the Nav2 navigation stack (Section 2.5), the physical Quanser QCar platform (Section 2.6), and the general concept of the sim-to-real gap (Section 2.7).

2.1 The ROS 2 Communication Model

The Robot Operating System (ROS) is not an operating system in the kernel sense but a middleware and tooling ecosystem for distributed robotics software: independent processes (*nodes*) communicate by publishing and subscribing to named, typed *topics*, calling *services* for synchronous request/response interactions, or invoking long-running, cancellable, feedback-reporting *actions* [9]. The original ROS (subsequently referred to as ROS 1) relied on a custom TCP-based transport and a single centralised **roscore** master process for node discovery – a design later recognised as unsuitable for real-time, multi-robot, or safety-relevant deployments, since the master is both a single point of failure and a source of unbounded discovery latency [4].

ROS 2 replaces this transport with the Data Distribution Service (DDS), an OMG-standardised, decentralised publish-subscribe protocol originally developed for defence and industrial real-time systems [7]. Each ROS 2 node embeds a DDS participant; nodes discover each other via a standardised discovery protocol (commonly Simple Discovery Protocol, SPDP/SEDP) rather than a central registry, and topic/service/action semantics are implemented on top of DDS’s publish-subscribe and request-reply patterns via the ROS Middleware interface (RMW), an abstraction layer that lets multiple DDS implementations (Fast DDS, Cyclone DDS, and others) serve as ROS 2’s transport interchangeably [6]. DDS communication is governed per-topic by Quality of Service (QoS) policies – reliability (**RELIABLE** vs. **BEST_EFFORT**), durability (**VOLATILE** vs. **TRANSIENT_LOCAL**, the latter retaining and re-delivering the last sample(s) to late-joining subscribers), and history depth among others – and a mismatch between a publisher’s and a subscriber’s QoS settings can silently prevent message delivery even when both sides otherwise appear correctly configured; Section 6.5.2 describes one such mismatch encountered directly during this project’s own diagnostic tooling.

This discovery and transport layer is directly relevant to this thesis’s central architectural constraint. DDS participants must share a compatible wire protocol version to discover and communicate with each other at all; Chapter 4 documents that the QCar’s factory-installed ROS 2 Dashing distribution (2019) and this project’s ROS 2 Humble development host (2022) do not, in practice, satisfy this requirement, which is the direct motivation for the TCP/JSON bridge presented there.

2.1.1 DDS Discovery: Why an Incompatibility Manifests as Silence

Understanding *why* a protocol-version mismatch produces total silence rather than a partial or degraded connection requires a closer look at how two DDS participants find each other in the first place, since this mechanism is what Chapter 4’s diagnosis directly exercises. DDS discovery happens in two stages. First, the Simple Participant Discovery Protocol (SPDP) has every participant periodically multicast (by default) a small announcement of its own existence and the network endpoints it listens on, and listen for the equivalent announcements from others – this is how two DDS processes on the same network first become aware of each other at all, with no central broker involved. Second, once two participants know of each other via SPDP, the Simple Endpoint Discovery Protocol (SEDP) exchanges the finer-grained information each side needs to actually match publishers to subscribers: topic names, data types, and QoS policies. Only after SEDP completes successfully does either side create the local proxy objects needed to actually send or receive user data – a topic that exists on one side but was never successfully matched via SEDP behaves, from the application’s point of view, exactly as if the other process did not exist, with no error raised on either side, since DDS discovery is a best-effort background process, not a connection attempt that can fail loudly.

Both SPDP and SEDP messages are themselves serialised using the DDS wire protocol (RTPS – Real-Time Publish-Subscribe), which is versioned. A participant running an RTPS implementation that predates a given protocol revision does not necessarily reject a newer participant’s announcement outright; depending on exactly which fields changed between versions, it can instead fail to parse it correctly, or parse it into a form that never completes the handshake – from the outside, this looks identical to no announcement having arrived at all. This is consistent with what Section 4.1 confirmed directly on hardware: raw UDP traffic reached both machines correctly (ruling out the network itself), but SPDP/SEDP discovery between ROS 2 Dashing’s and ROS 2 Humble’s respective bundled Fast DDS versions never completed regardless, exactly the signature of an RTPS-level incompatibility rather than a firewall or addressing problem.

QoS policies add a second, independent way for two otherwise discoverable endpoints to fail to exchange data, this time by *design* rather than by defect – DDS defines an explicit compatibility rule per policy, evaluated during SEDP matching, that a subscriber may request a QoS *no stronger* than what a publisher offers (Table 2.1). A subscriber requesting a policy stronger than the publisher offers on that dimension is a declared incompatibility, and DDS silently does not match that pair – again, an application-level developer would learn nothing about this without independently comparing the two sides’ QoS settings.

Table 2.1: QoS compatibility rule for the two policies relevant to this thesis (strongest requestable value shown first).

Policy	Publisher offers	Subscriber may request
Reliability	RELIABLE	RELIABLE or BEST_EFFORT
Reliability	BEST_EFFORT	BEST_EFFORT only
Durability	TRANSIENT_LOCAL	TRANSIENT_LOCAL or VOLATILE
Durability	VOLATILE	VOLATILE only

By this rule, a VOLATILE subscriber against a TRANSIENT_LOCAL publisher is formally com-

patible – which is exactly the configuration Section 6.5.2 found, on real hardware, delivering zero messages in practice despite being spec-compliant. Section 6.5.2 discusses the likely practical reason (durability governs whether *historical*, already-published samples are replayed to a late-joining subscriber, not merely whether future ones are delivered – a **VOLATILE** subscriber that joins after a transient-local publisher’s relevant samples were already sent may receive none of the backlog and then simply wait for a next sample that, for a low-frequency topic like action status, may not arrive for a long time); the point for this chapter is the general one – formal QoS compatibility, as defined by the DDS specification, is a necessary but not sufficient condition for an application to actually observe the message flow it expects.

2.1.2 The Action Interface and Goal Lifecycle

Nav2’s `navigate_to_pose` interface, used throughout Chapter 6, is a ROS 2 *action* – built on top of topics and services rather than being a distinct transport, an action pairs a goal-request service call with two topics the server publishes on: a feedback topic (periodic progress updates) and a status topic, which reports each known goal’s current state as one of a fixed enumeration (**ACCEPTED**, **EXECUTING**, **CANCELING**, **SUCCEEDED**, **CANCELED**, **ABORTED**, among others). The status topic uses **TRANSIENT_LOCAL** durability specifically so a subscriber that starts listening partway through a goal’s execution can still recover its current state from the retained history, rather than being blind until the next status change – the same durability property discussed above.

Because status updates are only published when the server’s internal state machine actually transitions, and the status topic’s history depth is finite, a fast state machine can advance through more than one transition between two publishes that an external observer actually receives – there is no guarantee every individual state is individually observable, only that the *latest* state at any polling instant is correct. Section 6.5.2 documents exactly this: a goal observed to skip directly from absent to **EXECUTING** in one status message, with no separately observable **ACCEPTED** update in between, even though the server’s internal state machine does pass through **ACCEPTED** on its way to **EXECUTING** – the transition happened, but no status publish was ever emitted while it was current in isolation. This is a general property of polling a periodically published state summary rather than a special case of this one topic.

2.2 Ackermann Steering Kinematics

The QCar is a rear-wheel-drive vehicle with front-wheel Ackermann steering: the two front wheels pivot independently about near-vertical steering axes such that, for any commanded steering angle, both front wheels’ instantaneous velocity vectors are perpendicular to lines drawn through a single common point on the extended rear axle – the vehicle’s instantaneous centre of rotation. This geometric constraint means the two front wheels must, in general, be steered to slightly different angles (the inner wheel turning more sharply than the outer) so that all four wheels roll without slipping through a common turn centre.

For control and estimation purposes, this thesis follows the standard simplification to a single-track (“bicycle”) model [10], which replaces the two front wheels with a single virtual wheel at the centre of the front axle steered at angle δ , and the two rear wheels with a single

virtual wheel at the centre of the rear axle. With vehicle heading θ , wheelbase L (the longitudinal distance between the front and rear axles), and rear-axle speed v , the kinematic bicycle model is:

$$\dot{x} = v \cos \theta \quad (2.1)$$

$$\dot{y} = v \sin \theta \quad (2.2)$$

$$\dot{\theta} = \frac{v}{L} \tan \delta \quad (2.3)$$

and the corresponding instantaneous turning radius about the rear axle is

$$R = \frac{L}{\tan \delta}. \quad (2.4)$$

Equation 2.3 is the basis of the onboard bridge’s dead-reckoning odometry described in Chapter 4, which genuinely integrates wheel-encoder and commanded-steering readings forward through this relationship at every control cycle to estimate pose from scratch. A single-track model is a simplification – it assumes no lateral tyre slip and treats both front wheels as a single steering angle rather than the true Ackermann-differentiated pair – and Chapter 5 presents direct evidence of where this simplification’s limits were reached in practice on physical hardware (Section 5.5.2).

2.2.1 The True Two-Wheel Ackermann Angle

The single-track model above is a control- and estimation-time convenience; it is not what the Gazebo drive plugin actually commands at the level of the two physical front wheels. Because the two front wheels steer about laterally separated axes, a single-track model’s one steering angle δ must, in general, be converted into two slightly different angles – the inner wheel (on the side the vehicle is turning toward) steered more sharply than the outer wheel – so that both front wheels’ rolling directions remain tangent to circles sharing the same centre on the extended rear axle, the condition for rolling without slipping introduced in Section 2.2’s opening paragraph. With track width T (the lateral distance between the two front steering axes) and wheelbase L , the exact left- and right-wheel steering angles corresponding to a single-track angle δ are:

$$\delta_{\text{left}} = \text{atan2}\left(\tan \delta, 1 - \frac{T}{2L} \tan \delta\right) \quad (2.5)$$

$$\delta_{\text{right}} = \text{atan2}\left(\tan \delta, 1 + \frac{T}{2L} \tan \delta\right) \quad (2.6)$$

(signed so that a positive δ , a left turn, gives $\delta_{\text{left}} > \delta_{\text{right}}$ – the inside wheel of the turn steers further). This is precisely the pair of relationships the `gazebo_ros_ackermann_drive` plugin evaluates every simulation step from a single commanded angular velocity, and is the formula Chapter 3 refers to when describing the plugin auto-computing Ackermann geometry from URDF joint positions: T and L in Equations 2.5–2.6 are measured directly from the URDF’s own front-hub joint origins at spawn time, never stated as separate configuration values. As $T \rightarrow 0$ (a vanishingly narrow front track), both angles converge to the single-track δ ,

confirming Equations 2.5–2.6 correctly generalise, rather than contradict, the simpler model of Equations 2.1–2.3.

Table 2.2: QCar Ackermann geometry parameters used throughout this thesis.

Parameter	Value	Source
Wheelbase L	0.257 25 m	URDF joint geometry (measured)
Front track width	0.1118 m	URDF joint geometry (measured)
Rear track width	0.1122 m	URDF joint geometry (measured)
Wheel radius	0.033 m	Wheel mesh bounding box (measured)
Max. steering angle δ_{max}	0.5236 rad (30°)	Vehicle/plugin limit
Min. turning radius (from Eq. 2.4)	0.446 m	Derived

2.3 Simulation with Gazebo

Gazebo is a physics-based 3D robot simulator that integrates a rigid-body dynamics engine, sensor models (LiDAR, cameras, IMUs), and a plugin architecture for injecting custom actuator and controller logic into the simulated world [3]. Two architecturally distinct generations of Gazebo are relevant to this project’s history: *Gazebo Classic* (versions up to 11), which integrates with ROS 2 through the separate `gazebo_ros` / `gazebo_plugins` packages, and the newer *Gazebo Sim* (formerly Ignition Gazebo), which integrates through `ros_gz_sim` / `ros_gz_bridge`. The two are not drop-in replacements for each other: plugin filenames, topic remapping mechanisms, and world-file (SDF) syntax for built-in physics/sensor systems all differ between them, and a package written against one will not load under the other without a systematic conversion (encountered directly in this project and reported in Section 3.3).

This project uses Gazebo Classic 11 with the `gazebo_ros` family of plugins, matching the ROS 2 Humble / Ubuntu 22.04 host environment, in which Gazebo Sim’s binaries are not available at all. The robot itself is described in the Unified Robot Description Format (URDF), an XML dialect specifying the vehicle’s kinematic tree (links and joints), visual meshes, and collision geometry; a `<gazebo>` block within the URDF attaches simulation-only elements – sensor definitions and the drive plugin (`libgazebo_ros_ackermann_drive.so`) that subscribes to `/cmd_vel` and converts commanded linear/angular velocity into per-wheel drive and steering-hub joint efforts using the Ackermann geometry of Section 2.2, which it auto-computes from the URDF’s own joint and collision geometry at spawn time rather than requiring it to be restated in a separate configuration file.

A recurring practical property of this plugin, directly relevant to Chapter 3, is that its wheel-radius auto-detection only supports primitive collision shapes (cylinders, spheres); a mesh-shaped wheel collision silently yields a wheel radius of zero, which propagates as a division-by-zero into the plugin’s velocity PID controller. Simulated friction is likewise not automatic: both the wheel links and the ground plane must be assigned explicit Coulomb friction coefficients (SDF’s `mu1/mu2`) or the simulated vehicle exhibits perfect wheel slip with no resulting translation regardless of commanded wheel speed. Neither failure mode raises an error – both look, from the outside, like “the robot is not moving,” and both are addressed in detail in Section 3.5.

2.3.1 The Plugin’s Control Loop: PID and the Physics Step

The drive plugin does not run on its own timer; it registers a callback on Gazebo’s `WorldUpdateBegin` event, which fires once per physics step of the rigid-body solver – by default 1 ms of simulated time per step, i.e. 1000 Hz, independent of and generally much faster than any ROS 2 topic rate in the system. Every single physics step, regardless of whether a new `/cmd_vel` message has arrived, the callback re-reads the rear wheel’s current angular velocity from the physics engine, compares it against the most recently *commanded* linear velocity converted to an equivalent target angular velocity ($\text{target_linear}/\text{wheel_radius}$), and passes that error through a discrete PID controller whose output is applied directly as a joint *force* (torque), not a velocity – this is a torque-controlled velocity-tracking loop, not a kinematic teleport of the wheel to a commanded speed. The controller update takes the classical form

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}, \quad e(t) = \text{measured}(t) - \text{target}(t), \quad (2.7)$$

evaluated once per physics step with the integral and derivative terms accumulated/estimated over the actual (possibly non-uniform) time elapsed since the previous step, and with the integral term optionally clamped to a configured $[i_{\min}, i_{\max}]$ range to prevent unbounded windup while the system is far from its target. Three independent instances of this controller run inside the plugin – one for the rear-axle drive velocity, and one each for the left and right steering hubs, tracking the two Ackermann angles of Equations 2.5–2.6 independently rather than a single shared steering command. All three sets of gains (K_p , K_i , K_d) are declared per-vehicle in the URDF’s `<gazebo>` plugin block, and Section 3.5 documents in detail what happens when these gains are tuned for one physical configuration (e.g. frictionless wheels) and then reused unchanged after that configuration changes.

Equation 2.7 also explains precisely why the wheel-radius defect above is destructive rather than merely inaccurate: with $\text{wheel_radius} = 0$, the target angular velocity $\text{target_linear}/\text{wheel_radius}$ is not merely wrong, it is unbounded for any nonzero commanded linear velocity, so $e(t)$ itself is unbounded and the PID controller is driven to its output saturation limit (or to a floating-point infinity/NaN) regardless of how the three gains are tuned – no amount of gain retuning can compensate for an error signal that is not a finite number in the first place, which is why this class of bug survives naive “just tune the PID harder” troubleshooting.

2.3.2 Contact Friction: the Coulomb Model and Friction Cone

Gazebo Classic’s default physics engine, the Open Dynamics Engine (ODE), models contact friction at each collision point using a Coulomb friction approximation: the maximum tangential (sliding) force a contact can sustain before slipping is proportional to the normal (compressive) force pressing the two surfaces together, with the constant of proportionality being the friction coefficient μ . ODE further allows this coefficient to be specified separately along two orthogonal directions in the contact plane (`mu1/mu2` in the URDF/SDF – e.g. a tyre could in principle grip differently along its rolling direction than sideways across it, though this project sets both equal, i.e. isotropic friction), and its exact treatment of the resulting two-dimensional friction limit is configurable between a computationally cheaper four-sided “friction pyramid” approximation

and a smoother `cone_model` approximation to the true circular Coulomb limit – the latter is explicitly selected in this project’s drive plugin (`physicsEngine->SetParam("friction_model", "cone_model")`), trading a small amount of solver cost for a more physically faithful friction limit shape.

Critically, **ODE applies no friction at all to a contact whose μ_1/μ_2 default to zero** – there is no implicit “reasonable default” friction coefficient assumed for an unconfigured surface, unlike, say, a default coefficient of restitution. This is precisely why Section 3.5’s bug 2 (zero wheel friction) produces literal, total 100% slip rather than merely reduced traction: with $\mu = 0$, the Coulomb limit on tangential force is zero regardless of how hard the wheel presses into the ground, so no sliding friction force – and therefore no forward thrust on the chassis – can develop at all, no matter how fast the wheel itself spins.

2.4 Simultaneous Localization and Mapping: Cartographer

Simultaneous Localization and Mapping (SLAM) is the problem of building a map of an unknown environment while concurrently estimating the robot’s pose within that same map, using only the robot’s own sensor stream and (optionally) odometry. This thesis uses Google’s Cartographer [2], a real-time 2D/3D LiDAR SLAM system that performs local scan matching to build submaps and a global pose-graph optimisation to detect and correct loop closures, exposed to ROS 2 via the `cartographer_ros` package. Cartographer can either fuse external odometry into its scan-matching prediction or perform scan-matching-only pose estimation (`use_odometry: false`), in which case it also provides the full `map` $\rightarrow$ `odom` $\rightarrow$ `base` transform (TF) chain itself (`provide_odom_frame: true`) rather than expecting an external odometry source to supply the `odom` $\rightarrow$ `base` portion. This configuration choice – made for this project’s mapping pipeline and detailed in Chapter 4 – turned out to remove an entire class of TF-ownership conflict between Cartographer and the vehicle’s own drive-plugin odometry that would otherwise arise, since ROS 2’s TF system permits only one broadcaster per transform edge at a time without producing ambiguous, jittering results.

2.5 The Nav2 Navigation Stack

Nav2 is the ROS 2 successor to the ROS 1 navigation stack, providing a modular, plugin-based pipeline for autonomous point-to-point navigation against a known (or concurrently built) map [5]. Its major components, all used in this thesis, are:

Costmaps Layered 2D occupancy grids (static map layer, obstacle/LiDAR layer, inflation layer) that represent both the known static environment and dynamically observed obstacles as traversability cost, separately for global path planning and local trajectory control.

AMCL (Adaptive Monte Carlo Localization) A particle-filter localisation algorithm that maintains a probability distribution over the robot’s pose as a weighted set of discrete pose hypotheses (particles), resampled according to how well each particle’s predicted LiDAR scan matches the actual scan against the static map, following the general Monte Carlo localisation framework [11]. AMCL’s motion-model noise parameters (commonly

denoted $\alpha_1\text{--}\alpha_5$) control how much particle spread is injected per unit of commanded motion; Chapter 6 documents a real localisation failure mode – and its correction – tied directly to these parameters.

Global planning Nav2’s `SmacPlannerHybrid` plugin, used in this project, plans a kinematically feasible path using a hybrid-state A* search with a Reeds-Shepp motion primitive set, respecting the vehicle’s minimum turning radius (Table 2.2) rather than planning paths the physical vehicle cannot actually follow.

Local trajectory control: MPPI Model Predictive Path Integral control [13, 12] is a sampling-based model-predictive control method: at each control cycle, a large number ($\mathcal{O}(1000)$) of candidate control sequences are simulated forward through a vehicle model over a fixed horizon, each rollout is scored by a weighted sum of cost terms (*critics* in Nav2’s terminology – path alignment, path following, goal distance, obstacle cost, and others), and the executed control is an importance-weighted average across all sampled rollouts rather than the single lowest-cost trajectory. This avoids the need for a convex, analytically differentiable cost function or vehicle model, at the cost of requiring many parallel forward simulations per control cycle. Nav2’s MPPI controller plugin supports custom critic plugins loaded alongside the stock set; Section 6.2 describes a custom critic developed as part of this project’s navigation stack.

Behavior Trees Nav2 orchestrates recovery behaviours (costmap clearing, spinning in place, waiting, backing up) and overall navigation logic via an externally configurable behaviour tree (BT) rather than hard-coded state machine logic, allowing per-robot customisation of, for instance, which recovery actions are physically valid for a given platform – directly relevant to Section 3.6, where the stock recovery tree’s default in-place `Spin` action is shown to be physically unachievable for an Ackermann-steered vehicle.

2.5.1 MPPI in Detail: Sampling, Scoring, and the Softmax Update

MPPI’s central idea is to replace the search for a single optimal control sequence with a weighted average over many randomly perturbed candidates, where the weighting itself is derived from an information-theoretic optimality argument rather than chosen heuristically [13, 12]. Concretely, given a current nominal control sequence $\mathbf{u} = (u_0, \dots, u_{T-1})$ over a horizon of T steps, the controller draws K independent perturbed sequences

$$u_t^{(k)} = u_t + \varepsilon_t^{(k)}, \quad \varepsilon_t^{(k)} \sim \mathcal{N}(0, \Sigma), \quad k = 1, \dots, K, \quad (2.8)$$

rolls each one forward through the vehicle model (the Ackermann bicycle model of Section 2.2, in this project’s configuration) to obtain a full predicted trajectory $\tau^{(k)}$, and evaluates a total cost $S(\tau^{(k)})$ for each – in Nav2’s implementation, this total cost is exactly the running sum that every enabled critic plugin (including the custom `EarlyCommitCritic` of Section 6.2) adds into via a single shared per-trajectory accumulator, so Section 6.2’s critic contributes one additive term among several to $S(\tau^{(k)})$, not a separate decision stage. Trajectories are then combined into an updated control sequence via an importance-sampling weighted average:

$$w^{(k)} = \frac{\exp\left(-\frac{1}{\lambda} \left(S(\tau^{(k)}) - \rho\right)\right)}{\sum_{j=1}^K \exp\left(-\frac{1}{\lambda} \left(S(\tau^{(j)}) - \rho\right)\right)}, \quad u_t^{\text{new}} = \sum_{k=1}^K w^{(k)} u_t^{(k)}, \quad (2.9)$$

where $\rho = \min_k S(\tau^{(k)})$ is subtracted purely for numerical stability (so the largest exponent evaluated is always zero) and $\lambda > 0$ is a temperature parameter controlling how sharply the weighting favours the very lowest-cost rollouts: as $\lambda \rightarrow 0$, Equation 2.9 approaches a hard arg-min over the sampled trajectories, while larger λ approaches a uniform average across all of them, largely ignoring cost. Only the first control u_0^{new} of the updated sequence is actually sent to the vehicle each control cycle, after which the whole horizon is shifted by one step and the process repeats – the standard receding-horizon pattern shared with other model-predictive control methods, but arrived at here via random sampling and a softmax-like weighted average rather than a gradient- or search-based optimisation over a differentiable cost surface. This is precisely why MPPI tolerates the sharply non-smooth, hand-crafted critic terms used throughout Nav2 (including a critic like `EarlyCommitCritic`, which is not differentiable in any convenient closed form) without requiring them to be reformulated as a smooth, gradient-friendly cost function first.

2.5.2 AMCL in Detail: The Particle Filter and the Recovery Mechanism

AMCL maintains its belief about the robot’s pose as a set of N weighted particles $\{(x^{(i)}, w^{(i)})\}_{i=1}^N$, each a concrete pose hypothesis, updated on every new odometry/LiDAR pair through the standard Monte Carlo localisation prediction-update-resample cycle [11]:

1. **Prediction (motion model).** Each particle is propagated forward using the commanded/odometric motion since the last update, corrupted by noise whose magnitude is controlled by the α_1 – α_4 parameters mentioned in the component list above (roughly: α_1/α_2 scale rotational noise from the odometry’s rotational and translational components respectively, α_3/α_4 scale translational noise the same way) – larger values inject more particle spread per unit of commanded motion, modelling greater distrust in odometry.
2. **Update (measurement model).** Each particle’s weight is set proportional to the likelihood of the actual LiDAR scan given that particle’s hypothesised pose and the static map, typically via a likelihood-field model that scores each scan endpoint by its proximity to the nearest mapped obstacle from that hypothesised pose.
3. **Resampling.** A new particle set is drawn, with replacement, with probability proportional to weight – particles consistent with the actual scan multiply, inconsistent ones are discarded, and the particle cloud statistically converges toward the true pose over repeated cycles. AMCL’s “adaptive” element (KLD-sampling) additionally varies N itself between update cycles, using fewer particles once the distribution is already tightly converged and more while it remains spread out.

The parameters directly responsible for the freeze failure mode documented in Section 5.2 are a separate pair from the four motion-model α s above: `recovery_alpha_slow/recovery_alpha_fast`

govern an *augmented* MCL recovery mechanism, tracking two exponential moving averages of the particle set’s average weight – a slow long-run average $\bar{w}_{\text{slow}}$ and a fast short-run average $\bar{w}_{\text{fast}}$, updated after every measurement step as

$$\bar{w}_{\text{slow}} += \alpha_{\text{slow}} (\bar{w} - \bar{w}_{\text{slow}}), \quad \bar{w}_{\text{fast}} += \alpha_{\text{fast}} (\bar{w} - \bar{w}_{\text{fast}}), \quad (2.10)$$

where $\bar{w}$ is the current cycle’s average particle weight. During resampling, instead of drawing every new particle from the existing weighted set, each new particle is drawn *uniformly at random from the entire map* with probability $\max(0, 1 - \bar{w}_{\text{fast}}/\bar{w}_{\text{slow}})$ – intuitively, if recent particle quality ($\bar{w}_{\text{fast}}$) has dropped noticeably below the longer-run baseline ($\bar{w}_{\text{slow}}$), the filter injects fresh, unbiased hypotheses to help it recover from a bad convergence or a “kidnapped robot” scenario, rather than continuing to resample only from a particle set that may have already converged to the wrong pose. Setting $\alpha_{\text{slow}} = \alpha_{\text{fast}} = 0.0$ – the value this project’s simulation-tuned configuration originally inherited – means neither running average is ever updated from its initial value, so this random-injection probability is permanently fixed rather than responsive to real degradation in particle quality: on real, non-ideal sensor noise (unlike a clean simulation, where the filter rarely needs this recovery path at all), the particle set can converge overconfidently to an incorrect pose with no mechanism available to reintroduce diversity, which is the permanent freeze Section 5.2 reports and fixes by raising these two parameters off zero.

2.6 The Quanser QCar Platform

The physical platform underlying this thesis is the Quanser QCar, a 1/10th-scale rear-wheel-drive, Ackermann-steered research vehicle [8]. Its onboard compute is an NVIDIA Jetson TX2 system-on-module (running Ubuntu 18.04 LTS with 32 GB eMMC storage and Python 3.6.9), which ships with both ROS 1 Melodic and ROS 2 Dashing pre-installed side by side, neither of which is auto-sourced in the default shell environment. Sensing is provided by a 2D LiDAR and an onboard IMU; drive and steering actuation are exposed through Quanser’s own Hardware Abstraction Layer / Product Application Layer (HAL/PAL) Python library, which wraps the vehicle’s motor, servo, encoder, and battery-monitoring hardware behind a `QCar` object exposing `read()/write()` calls and a small number of documented safety limits: PWM duty cycle is saturated to $\pm 30\%$ with a $100\%/s$ rate limiter, the steering command range is ± 0.5 rad against a physical limit of ± 0.5236 rad ($\pm 30^\circ$), and an overcurrent protection state (triggered at $5\text{ A}/8\text{ s}$, $10\text{ A}/2\text{ s}$, or $15\text{ A}/0.5\text{ s}$) forces the vehicle into a neutral mode until the underlying hardware-in-the-loop device is restarted. These limits are respected throughout every piece of drive-control code presented in this thesis.

A property of the HAL library specific to this platform, and central to one of this thesis’s most significant findings (Section 5.6), is its two supported I/O modes: *immediate I/O* (`readMode=0`), where each call to `read()` triggers a fresh hardware sample, and *task-based I/O* (`readMode=1`), where a background acquisition task begins sampling into a fixed-depth ring buffer at object construction time, independently of when (or whether) the caller ever calls `read()`. This distinction is not merely an implementation detail; Chapter 5 documents how it produced a measurement artefact large enough, and stable enough, to be repeatedly and reasonably

misattributed to genuine vehicle physics before its true cause was found.

2.7 The Sim-to-Real Gap

The term *sim-to-real gap* (or *reality gap*) refers, broadly, to any systematic discrepancy between a robot’s simulated and physical behaviour under nominally identical software and commands [14]. Sources of this gap are commonly grouped into a few broad categories, all of which recur, in concrete and specific form, across this thesis’s methodology chapters:

- **Unmodelled or under-modelled physical effects** – e.g. static friction / stiction, actuator response lag, or non-linear actuator-to-output mappings that a simulator either omits entirely or models only in idealised form. Chapter 5’s throttle calibration and friction-deadband findings are a direct instance of this category.
- **Sensor and estimation differences** – simulated sensors are frequently noiseless or near-noiseless and perfectly time-synchronised with the simulation clock, whereas physical sensors carry real noise, real latency, and, in a distributed system, real inter-process clock skew. Chapter 4’s clock-offset and LiDAR angle-convention fixes fall into this category.
- **Software/communication-layer differences** – a simulated robot typically runs its entire software stack as co-located ROS 2 processes on one machine; a physical robot may be split across incompatible middleware versions, unreliable wireless links, or (as found directly in this project) firmware- or driver-level buffering that a simulation has no equivalent of at all. Chapters 4 and 5 both document findings in this category.

A methodological point that recurs throughout this thesis, and is made explicit here because it shapes every later chapter’s approach, is that these categories are not always distinguishable from the symptom alone. A fixed-duration, speed-independent delay between a software command and its physical effect is, from a black-box measurement standpoint, equally consistent with drivetrain coasting (a physical-effect explanation), a command-delivery lag in the bridge software (a communication-layer explanation), or, as this thesis’s own investigation ultimately found, a stale-data artefact in a sensor read path (a sensor/estimation-layer explanation that happens to mimic a physical one almost exactly). Distinguishing between these required, in every case documented in this thesis, going beyond the symptom to a direct measurement or source-level inspection of the specific layer under suspicion – a theme Chapter 5 returns to explicitly.

3 Simulation Stack Development

This chapter documents the development of the simulation-side navigation stack that Chapters 4 through 6 later migrate onto physical hardware. The stack went through two full drive-architecture migrations before reaching the form used for hardware bridging (Sections 3.1–3.4), and along the way surfaced a middleware-level parser defect (Section 3.2.2) and a four-part vehicle-dynamics bug pattern that recurred, largely unchanged, across two sibling packages (Section 3.5). The chapter closes with a behaviour-tree fix specific to Ackermann-steered vehicles (Section 3.6).

3.1 Baseline: Manual Bicycle-Model Inversion

The navigation stack’s earliest drive architecture used two independent `ros2_control` `JointGroup` controllers (`drive_controller` for wheel velocity, `steering_controller` for hub position) driven by a hand-written node, `cmd_vel_to_drive.py`, that manually inverted the bicycle-model relationship of Equations 2.1–2.3 to compute a per-wheel velocity and steering-hub angle from an incoming `geometry_msgs/Twist` on `/cmd_vel`. This architecture worked, but duplicated the Ackermann inverse-kinematics logic that a purpose-built controller would otherwise provide, and – more importantly for a navigation stack meant to reach physical hardware – could not support Nav2’s recovery behaviours that require reversing, since the manual node had no reverse-driving logic of its own.

3.2 First Migration: `ros2_control`’s `AckermannSteeringController`

The first architectural migration replaced both custom `JointGroup` controllers and `cmd_vel_to_drive.py` with `ros2_control`’s own `ackermann_steering_controller` / `AckermannSteeringController` plugin, configured via `config/qcar_controllers.yaml`. This controller performs the same Ackermann inverse kinematics internally, and additionally publishes its own wheel-based odometry – which this project deliberately adopted as the vehicle’s `/odom` source in place of Gazebo’s `gazebo_ros_p3d` ground-truth plugin (the latter was kept running, but remapped to `/ground_truth/odom` for debugging comparison only). This was a considered choice, not a default: ground-truth odometry is trivially available in simulation but has no equivalent on the physical robot, whose odometry can only ever be a real, drift-prone dead-reckoning estimate (Section 4.4). Developing and tuning the navigation stack against the same *kind* of odometry it would have to work with on hardware, from the start, avoided deferring that discrepancy to the eventual hardware migration.

Because Nav2, AMCL, and `bt_navigator` all hard-code an expectation that odometry arrives on `/odom` and driving commands are consumed from `/cmd_vel`, and the new controller instead exposes these under its own namespaced topics (`~/odometry`, `~/reference_unstamped`), two `topic_tools` `relay` nodes were added to the launch files to bridge the naming mismatch without modifying Nav2 itself.

3.2.1 A Wheel-Axis Bug Exposed by the Migration

The migration exposed a latent URDF defect that had been silently compensated for by the old manual node and had therefore never produced a visible symptom before. The rear-left drive wheel joint (`base_wheelrl_joint`) had its `<origin>` yawed by 180° for correct mesh mirroring, but retained the *same* `<axis>` direction as the corresponding right-side joint – which, combined with the yaw flip, silently inverted that wheel’s effective rotation sense relative to a positive commanded velocity. The old `cmd_vel_to_drive.py` had manually compensated for this by commanding `[-speed, +speed]` to the left/right wheel velocities respectively; the new controller had no such compensation, and computed wheel odometry that did not match the vehicle’s real motion direction. The fix – correcting the `<axis>` declaration on the affected joint directly, rather than perpetuating a sign-flip workaround downstream in every consumer of that joint – illustrates a pattern worth stating explicitly: a URDF geometry defect that is silently absorbed by one consumer’s ad hoc compensation will resurface, often with a confusing symptom, the moment that consumer is replaced.

Listing 3.1 shows the two rear drive-wheel joints as they stand in the corrected URDF: the left joint’s mesh-mirroring yaw (`rpj="0 0 3.14"`) is paired with a negated `<axis>` so that a positive commanded wheel velocity turns both wheels in the same physical sense, whereas the right joint needs no such compensation since its origin carries no mirroring rotation.

```
1 <joint name="base_wheelrl_joint" type="continuous">
2   <parent link="base" />
3   <child link="wheelrl" />
4   <origin rpy="0 0 3.14" xyz="-0.12765 0.05610 0.03338" />
5   <!-- axis negated to compensate for the 180deg yaw above (mesh mirror) -
6       without this the left wheel spins opposite the right wheel for the
7       same signed velocity command -->
8   <axis xyz="0 -1 0" />
9 </joint>
10
11 <joint name="base_wheelrr_joint" type="continuous">
12   <parent link="base" />
13   <child link="wheelrr" />
14   <origin rpy="0 0 0" xyz="-0.12765 -0.05610 0.03338" />
15   <axis xyz="0 1 0" />
16 </joint>
```

Listing 3.1: Rear-left and rear-right drive-wheel joints (`urdf/qcar_model.xacro`) – the axis on the mirrored left joint is negated to compensate for its 180° origin yaw.

Why the sign flip happens: axes live in the joint’s own, rotated frame. The mechanism behind this bug is a direct consequence of one URDF convention that is easy to overlook: a joint’s `<axis>` vector is expressed in the joint’s *own* local frame – the frame produced by applying that same joint’s `<origin>` rotation to its parent link’s frame – not directly in the parent link’s (here, `base`’s) frame. The rear-right joint’s origin carries no rotation (`rpj="0 0 0"`), so its joint frame is identical to `base`’s frame, and `axis="0 1 0"` means exactly $+Y$ in the vehicle’s own reference

frame. The rear-left joint's origin, in contrast, is yawed 180° about Z (`rpj="0 0 3.14"`) so its mesh renders mirrored on the correct side of the chassis. A 180° yaw about Z maps a frame's own X and Y axes to their negatives while leaving Z unchanged ($X \rightarrow -X$, $Y \rightarrow -Y$), so that joint's *local* $+Y$ direction already points along *base*'s $-Y$ direction before the `<axis>` element is even considered. Declaring `axis="0 1 0"` on the rear-left joint – the value that looks, by simple inspection, like “the same axis as the right wheel” – therefore actually commands rotation about *base*'s $-Y$, the opposite physical sense from the right wheel's *base*-frame $+Y$, for the identical signed joint velocity. Negating it to `axis="0 -1 0"` (Listing 3.1) cancels the already-applied yaw, restoring both wheels to rotating about the same *base*-frame axis for the same commanded sign – the fix is not “pick whichever sign makes it drive forward” by trial and error, but a direct, derivable consequence of the joint's own origin rotation, which is why it generalises immediately to any other mirrored-mesh joint pair in this or a similar URDF.

3.2.2 A URDF Parser Defect in gazebo_ros2_control

While adding explanatory comments to the URDF for the wheel-axis fix above, `controller_manager` began failing to start altogether – with all downstream symptoms that implies (`joint_state_broadcaster` and the new controller never spawning, `/odom` never appearing, and no `map` TF being published either, since nothing in the chain that would produce it ever started). The failure was silent: no controller spawner logged an error explaining *why* it timed out waiting for `controller_manager`.

Isolating the cause required a delta-debugging-style bisection of the URDF file against the actual parsing code path, since multi-line comments, apostrophes, and file length were all plausible-looking red herrings that turned out not to matter. The root cause was found in how `gazebo_ros2_control` starts its internally-managed `controller_manager` node: the plugin passes the entire processed URDF as a single `-param robot_description:=<xml>` command-line-style argument, which `rcl`'s parameter-override grammar parses looking for a `node_name:param_name` pattern. That grammar treats *any* colon immediately followed by a space – `:` – anywhere in the argument string as a potential separator, including inside an XML comment that a human reader would never consider part of the “real” content. A direct reproduction using `rclpy`'s own argument parser confirmed the mechanism precisely: `"hello: world"` fails to parse while `"hello:world"` (no space) succeeds, and `package://URIs` or `foo:=bar` remappings are unaffected since neither places a bare space directly after a colon.

This is not specific to this project's URDF; it is a property of any `gazebo_ros2_control`-based setup, and is recorded here as a general finding: **a URDF or xacro file must never contain the literal sequence colon-followed-by-space, including inside comments**, or `controller_manager` fails to start with no diagnostic pointing at the actual cause.

3.3 Porting the Navigation Stack to ROS 2 Humble / Gazebo Classic

A sibling package in this project's workspace had been authored against ROS 2 Jazzy and the newer Gazebo Sim (Harmonic) simulator – the `gz sim` binary family, `ros_gz_sim` / `ros_gz_bridge` integration, and `gz-sim-*-system` world plugins – none of which are available on this project's ROS 2 Humble / Ubuntu 22.04 / Gazebo Classic 11 host. Rather than install a second, newer

simulator stack alongside the one already validated and working for the rest of this thesis’s packages (Section 3.2), the package was converted to Gazebo Classic, matching every other package in this project. Table 3.1 summarises the conversion, which generalises directly to any future package encountered that was authored against Gazebo Sim / Jazzy-era tutorials.

Table 3.1: Gazebo Sim (Jazzy) to Gazebo Classic (Humble) translation table.

Gazebo Sim (source)	Gazebo Classic (target)	Notes
<code>ros_gz_sim</code> / <code>ros_gz_bridge</code> (package.xml)	<code>gazebo_ros</code> / <code>gazebo_plugins</code>	
<code>gpu_lidar</code> sensor + gz-sim system plugin	<code>type="ray" + libgazebo_ros_ray_sensor.so</code>	
gz-sim imu sensor	classic imu + <code>libgazebo_ros_imu_sensor.so</code>	
gz-sim-joint-state-publisher-system	removed entirely	Not needed; the drive plugin’s own <code>publish_wheel_tf</code> broadcasts wheel/steering TF directly
gz-sim-ackermann-steering-system	<code>libgazebo_ros_ackermann_drive.so</code>	Classic plugin auto-computes wheel-base/track/radius from URDF geometry; the gz-sim system required stating them explicitly
5× <code><plugin filename="gz-sim-*-system"></code> (world SDF)	deleted entirely	Classic’s <code>gzserver</code> has physics, scene broadcast, and sensors built into its core, not opt-in systems
gz sim + <code>ros_gz_sim create spawn +</code>	<code>gazebo/gzserver</code> + <code>gazebo_ros</code>	No bridge node needed; classic plugins publish onto ROS 2 topics directly
<code>ros_gz_bridge</code> node	<code>spawn_entity.py</code>	
<code>GZ_SIM_RESOURCE_PATH</code> env-var for mesh resolution	not needed	<code>gazebo_ros</code> registers its own <code>package:// resolver</code>

One element of the original file required *no* change: the `<gazebo reference="link">` `<material>Gazebo/X</material></gazebo>` material-script blocks are classic-Gazebo-native syntax that happened to already be present and compatible, a reminder that a systematic conversion still benefits from checking each element rather than assuming every gz-sim-authored construct needs translating.

3.4 Second Migration: The Direct Drive Plugin

The navigation stack’s final drive architecture – the one this thesis’s hardware bridge (Chapter 4) targets – drops `ros2_control` entirely in favour of driving the robot through `libgazebo_ros_ackermann_drive.so` directly, declared in a `<gazebo>` block in the URDF rather than through a separate controller-manager configuration file, matching the approach already adopted for the sibling package during the Humble conversion above. This plugin subscribes to `/cmd_vel` and publishes `/odom` and the `odom` → `base` TF transform directly, again auto-computing wheelbase, track width, and wheel radius from URDF joint and collision geometry at spawn time – removing an entire category of configuration drift risk where a controller’s stated geometry parameters could silently fall out of sync with the URDF’s actual geometry, which had been possible (if not observed) under the first migration’s `ros2_control` YAML-based configuration. This is the architecture described in Section 2.3’s account of the

drive plugin, and the one whose real-world calibration Chapter 5 presents.

3.4.1 A Quiet Regression: Odometry Reverts to Ground Truth

This migration has one consequence worth stating explicitly, because it is easy to overlook and was not caught at the time: it silently reversed the odometry-source decision made in Section 3.2. Reading `gazebo_ros_ackermann_drive`’s own `UpdateOdometryWorld()` implementation directly shows that its `/odom` is computed from `model_->WorldPose()` and `model_->WorldLinearVel()` – the physics engine’s own exact, noise-free simulated pose and velocity for the whole vehicle body – not an integration of individual wheel-joint velocities or steering angles at all. In other words, the final simulation-side drive architecture publishes `/odom` that is, mechanically, no different from what `gazebo_ros_p3d` (Section 3.2) would have provided: perfect, driftless ground truth, despite carrying the same topic name and message type the first migration deliberately chose *instead of* ground truth.

This is not a bug in the sense the rest of this chapter uses the word – the plugin behaves exactly as its own source dictates, and nothing in this project’s testing depended on `/odom` actually drifting in simulation. It is, however, a genuine and previously unrecorded discrepancy between this thesis’s stated intent (train against the same *kind* of odometry the physical robot will have) and what the final simulation configuration actually exercises, and it is worth stating plainly rather than leaving the earlier claim uncorrected: **only the physical QCar’s own odometry (Chapter 4, Section 4.4) is genuine encoder-based dead reckoning in this project; the simulation’s final `/odom` is ground truth, exactly like the `gazebo_ros_p3d` source it was originally meant to replace.** Framed in terms of Section 2.7’s sim-to-real gap taxonomy, this means the localisation/odometry accuracy gap between simulation and hardware in this project is not purely an empirically discovered discrepancy – it is, in part, structurally guaranteed by this architecture, since the simulated AMCL/Nav2 stack is given a perfect motion estimate that its real-hardware counterpart can never have. Any future comparison of localisation accuracy between simulation and hardware should account for this rather than treating the two `/odom` streams as like for like.

3.5 The “Robot Not Moving Forward” Bug Pattern

Both the sibling package’s Humble conversion (Section 3.3) and this project’s own second migration (Section 3.4) independently produced the same surface symptom immediately afterward: the simulated vehicle did not drive forward under a sustained `/cmd_vel` command. In both cases, this single symptom turned out to be caused by a stack of independent bugs, each one masking the next until the previous was fixed – a pattern significant enough, and confirmed to recur closely enough between the two packages, to warrant treating as a general defect signature for this plugin/vehicle combination rather than two unrelated incidents. Table 3.2 summarises all bugs found across both occurrences.

Listing 3.2 shows the corrected form of one rear wheel link and its friction declaration, combining the fixes for bugs 1 and 2 from Table 3.2: an explicit cylinder `<collision>` geometry (the `<visual>` mesh, omitted here, is unaffected and keeps using the STL), and a separate `<gazebo reference=“...”>` block assigning Coulomb friction coefficients to that same link –

Table 3.2: The recurring “not moving forward” bug pattern, as found in both affected packages.

#	Bug	Symptom / mechanism
1	Wheel <code><collision></code> was the visual STL mesh, not a primitive shape	The plugin’s radius auto-detection only supports cylinder/sphere collisions; a mesh collision silently returns radius 0, so <code>target_linear / wheel_radius</code> becomes infinite, poisoning the velocity PID regardless of gain. Fixed by keeping the STL for <code><visual></code> but using an explicit <code><cylinder radius="0.036" length="0.0245"/></code> for <code><collision></code> .
2	Zero Coulomb friction on the wheel links	Even with a correct wheel radius, the wheel spins at exactly the commanded angular velocity forever while the chassis never translates – 100% slip. Fixed with <code><mu1>1.0</mu1><mu2>1.0</mu2></code> on every wheel link.
3	Absurd ground-plane friction (<code>mu=100, mu2=50</code>)	Once wheel friction is fixed, this combination produces a real physics instability/NaN blow-up on contact. Fixed by lowering to a realistic <code>mu=1, mu2=1</code> .
4	Razor-thin wheel/ground clearance (only ~ 0.38 mm geometric overlap at the original 0.033 m collision radius)	Enough overlap to rest on the ground without falling through, but not enough consistent contact-normal force to generate real traction even at correct friction – chassis twist stayed near zero despite full wheel spin. Fixed by padding the collision cylinder radius to 0.036 m (+3 mm), which alone flipped the vehicle from “spins in place” to sustained forward motion.

URDF has no wheel-friction concept of its own, so this Gazebo-specific extension is the only place that value is set at all.

```

1 <link name="wheelr1">
2   <collision>
3     <origin rpy="1.5708 0 0" xyz="0 0 0" />
4     <geometry>
5       <cylinder radius="0.036" length="0.0245" />
6     </geometry>
7   </collision>
8   <!-- <visual> (STL mesh) and <inertial> omitted here, unaffected -->
9 </link>
10
11 <!-- wheel-ground friction - without this the wheels spin freely with 100%
12     slip and the chassis never translates, regardless of PID gains -->
13 <gazebo reference="wheelr1">
14   <mu1>1.0</mu1>
15   <mu2>1.0</mu2>
16 </gazebo>

```

Listing 3.2: Wheel collision cylinder and friction (`urdf/qcar_model.xacro`) – fixes for bugs 1 and 2 in Table 3.2.

Mechanism: how bugs 1 and 2 poison the control loop and physics, in numbers.

Section 2.3.1 and Section 2.3.2 give the general theory bugs 1 and 2 exercise; concretely, for this vehicle, a commanded 0.3 m/s with a mesh-shaped (bug 1, radius $\rightarrow 0$) rear-wheel collision computes a target angular velocity of $0.3/0 \rightarrow \infty\text{ rad/s}$ in Equation 2.7’s error term – not a large but finite number that a differently-tuned PID could still track, but a value with no finite representation at all, driving the linear-velocity PID’s output straight to its saturation limit on the very first physics step regardless of gain. With the corrected 0.036 m radius (Listing 3.2) that same command instead computes a finite target of $0.3/0.036 \approx 8.33\text{ rad/s}$, a well-posed, trackable error signal. Bug 2 (zero friction) then governs whether that correctly-spinning wheel produces any chassis motion at all: with $\mu_1 = \mu_2 = 0$, Section 2.3.2’s Coulomb limit permits zero tangential contact force regardless of how much normal force the vehicle’s weight presses the wheel into the ground with, so the wheel can spin at exactly the commanded 8.33 rad/s indefinitely while contributing zero net force to the chassis – the two bugs therefore fail at two different, independently necessary conditions (a well-posed control error, and a nonzero force limit to actually convert correct wheel spin into chassis thrust), which is why fixing either one alone still left the vehicle stationary and only fixing both together produced motion.

The first occurrence of this pattern also included a preceding, more basic defect specific to that package: its mesh URIs (`package://.../models/qcar/QCarBody.stl`-style paths) pointed at a directory layout copy-pasted from this project’s own package rather than the sibling’s actual `meshes/` layout and file names, so the robot had *no* collision geometry at all and fell through the simulated ground plane – confirmed directly via `/odom` tracking a z -position reaching approximately $-38,000\text{ m}$. This is worth recording as a separate, more fundamental lesson: Gazebo Classic’s URDF-to-SDF loader silently rewrites `package://` mesh URIs to `model://`,

which only resolves via the `GAZEBO_MODEL_PATH` environment variable – and since neither package’s `package.xml` exports the `<gazebo_ros gazebo_model_path=.../>` tag required for that resolution, correcting the path string alone would not have been sufficient. The fix instead used xacro’s `$(find)` substitution to build `file://` URIs directly, which bypass package-path resolution entirely.

The second occurrence additionally required a numerical-stability fix not initially anticipated: wheel rotational inertia values that were physically correct for a solid disk of the wheel’s real mass and radius, but small enough in absolute terms to interact badly with the plugin’s 100 Hz PID control loop, producing an `Ogre::AxisAlignedBox::setExtents` assertion crash a few seconds into driving. Bumping wheel inertia roughly $20\times$ (`ixx=izz=0.0012`, `iyy=0.0022`) together with lowering the steering and linear-velocity PID gains resolved the crash; this value combination, once found working in the first occurrence, was reused directly in the second and confirmed stable there as well.

A further gain-tuning finding, specific to the second occurrence and worth stating as a general caution for reusing PID values across otherwise-identical vehicle configurations: a steering PID gain verified stable and accurate on a *frictionless* wheel configuration produced a real turning radius roughly $6\times$ larger than commanded once realistic wheel friction (bug 2 above) was added, because the same low gain that was previously sufficient (with no resistive torque to overcome) could no longer turn the front hubs against a now-gripping, rolling tyre. The fix – raising `left/right_steering_pid_gain` from 0.02 to 3.0 – was verified directly via TF lookups on the steering-hub joints during a commanded 1.0 m-radius turn, confirming hub angles converging to the correct $\sim 15^\circ$ – 18° Ackermann inner/outer split, and via a circle fit to the resulting odometry trace, which recovered a turning radius of 0.87 m against the 1.0 m commanded – a reasonable margin given the real-vs-ideal gap between the single-track model of Section 2.2 and the vehicle’s true four-wheel geometry.

3.5.1 Diagnostic Method

A methodological point generalises from both occurrences of this bug pattern: none of these four bugs are visible from `ros2 topic list` or a static inspection of the URDF alone. A simulated robot that appears correctly configured can simultaneously be in freefall, frozen inside a crashed-but-still-running `gzserver` process, or driving its wheels at 100% slip with zero real chassis motion – and none of these states are distinguishable without directly comparing `/odom` position over several seconds of sustained commanded motion, cross-checked against the Gazebo server log for silent `[Err]`/assertion lines. This became this project’s standard verification method for any subsequent URDF or physics change to the simulation stack.

3.6 A Behaviour-Tree Fix for Ackermann Recovery

How a behaviour tree actually executes. A behaviour tree is evaluated by repeatedly “ticking” it from the root: each tick propagates down the tree and every node it reaches returns one of `SUCCESS`, `FAILURE`, or `RUNNING` (still in progress, re-tick next cycle) back up to its parent, and how a parent node interprets its children’s results depends entirely on that parent’s type. A `RoundRobin` node – the one this section’s fix concerns – ticks exactly one of its children per

activation (advancing to the next child, wrapping back to the first, only once the currently-selected child itself returns `SUCCESS` or `FAILURE` rather than `RUNNING`), cycling through its full child list one at a time across repeated recovery attempts rather than trying all of them on a single activation. A `ReactiveFallback` (the parent of `RecoveryActions` in Listing 3.3) instead re-ticks its children from the first one every single tick, moving on to the next only if the current one fails – the two control-flow types are not interchangeable, and Nav2’s stock recovery subtree pairs them deliberately: the outer `ReactiveFallback` re-checks whether a goal update has arrived (via the `GoalUpdated` condition node) before committing to whichever recovery action the inner `RoundRobin` has advanced to.

A separate, functional (rather than purely physics-level) issue appeared once the vehicle was reliably driving: Nav2’s default recovery behaviour tree round-robins through `[ClearCostmaps, Spin(1.57 rad), Wait(5 s), BackUp(0.3 m)]` whenever the active controller reports it cannot make further progress. The `Spin` recovery commands a pure in-place rotation – physically impossible for an Ackermann-steered vehicle, whose steering angle has no effect while linear velocity is zero, so the vehicle simply cannot rotate without also translating. Roughly one in three recovery attempts was therefore a guaranteed no-op that burned its full timeout without attempting any corrective motion.

The fix was to remove `<Spin>` from the recovery action list in local copies of Nav2’s stock behaviour-tree XML files (`navigate_to_pose_w_replanning_and_recovery.xml` and its through-poses equivalent), wired into the launch configuration via a `RewrittenYaml` substitution. This required one non-obvious step: `RewrittenYaml` only overwrites *existing* leaf keys in a YAML file, it does not add new ones, so the parameter keys pointing at these custom BT file paths (`default_nav_to_pose_bt_xml` / `default_nav_through_poses_bt_xml`) had to already exist as placeholder entries in `nav2_params.yaml` before the launch-time substitution could take effect.

Listing 3.3 shows the resulting recovery fallback from the local behaviour-tree copy: a `RoundRobin` of clearing both costmaps, waiting, and backing up – with no `<Spin>` node present at all, compared to the stock tree’s four-way rotation through `[ClearCostmaps, Spin, Wait, BackUp]`.

```

1 <ReactiveFallback name="RecoveryFallback">
2   <GoalUpdated/>
3   <RoundRobin name="RecoveryActions">
4     <Sequence name="ClearingActions">
5       <ClearEntireCostmap name="ClearLocalCostmap-Subtree"
6         service_name="local_costmap/clear_entirely_local_costmap"/>
7       <ClearEntireCostmap name="ClearGlobalCostmap-Subtree"
8         service_name="global_costmap/clear_entirely_global_costmap"/>
9     </Sequence>
10    <Wait wait_duration="5"/>
11    <BackUp backup_dist="0.30" backup_speed="0.05"/>
12  </RoundRobin>
13 </ReactiveFallback>

```

Listing 3.3: Recovery actions in the local behaviour-tree copy (`config/nav2/behavior_trees/navigate_to_pose_w_replanning_and_recovery.xml`) – no `<Spin>` node, unlike the Nav2 stock tree.

Because `RoundRobin` advances exactly one position per activation, dropping `Spin` from a four-child list to this three-child list is not merely “the useless action happens less often” – it removes the guaranteed no-op activation entirely, so every single recovery activation now attempts a physically achievable action, and the cycle back to any one specific action (e.g. back to `BackUp` after it was last tried) also completes one activation sooner than it did with four children in the rotation.

3.7 Summary of the Simulation-Stack Architecture

By the end of this chapter’s work, the simulation-side navigation stack consists of: a URDF-described vehicle driven directly by `libgazebo_ros_ackermann_drive.so` (Section 3.4), publishing ground-truth-derived `/odom` (Section 3.4.1 – a quiet regression from the genuine dead-reckoning odometry the first migration, Section 3.2, had deliberately adopted), with corrected wheel collision geometry, friction, and inertia (Section 3.5), tuned PID gains for both drive and steering, and a custom Ackermann-aware recovery behaviour tree (Section 3.6). This is the configuration Chapter 4 treats as the reference to be matched – as closely as the physical robot’s own constraints allow – when bridging to hardware.

4 Real-Hardware Bridge Architecture

This chapter presents the architecture built to connect the simulation-validated navigation stack of Chapter 3 to the physical QCar. Section 4.1 establishes why native ROS 2 communication with the robot’s onboard compute is not available, which motivates the TCP/JSON bridge architecture of Section 4.2. The remaining sections document the sensor and actuator correctness issues found while bringing that bridge up to the same fidelity as the simulation stack: LiDAR angle convention (Section 4.3), odometry (Section 4.4), steering trim (Section 4.5), and inter-machine clock synchronisation (Section 4.6).

4.1 The ROS 2 Dashing / Humble Incompatibility

As described in Section 2.6, the QCar’s onboard Jetson TX2 ships with ROS 2 Dashing (released 2019) pre-installed, while this project’s navigation stack targets ROS 2 Humble (released 2022). Section 2.1 established that ROS 2 nodes communicate via DDS participants that must share a compatible discovery and wire protocol to find each other at all; the working hypothesis at the start of this thesis’s hardware phase was that this would be a soft compatibility problem – degraded performance, perhaps, but not a hard failure.

This was tested directly, not assumed. A minimal ROS 2 publisher (`demo_nodes_cpp talker`) on the QCar and a minimal subscriber (`demo_nodes_py listener`) on the development PC, both configured with the same `ROS_DOMAIN_ID`, never established communication across three repeated attempts, with the talker independently confirmed to be actively publishing each time. The network layer itself was systematically ruled out before concluding this was a middleware-level problem: raw UDP unicast passed cleanly in both directions, including on the exact Fast DDS discovery port for the domain in use; no firewall was active on either host; and an explicit unicast-peers Fast DDS XML profile, which bypasses multicast-based discovery entirely, still failed to achieve discovery. This isolates the failure to the DDS protocol implementation itself – a genuine Fast DDS protocol-version incompatibility between the versions bundled with ROS 2 Dashing and ROS 2 Humble respectively [1], not a network, firewall, or configuration problem that could be worked around within ROS 2 itself.

This result rules out every architecture that assumes native ROS 2 interoperability between the two machines, including running a Humble-compatible container on the QCar: a Humble Docker container was considered early on, but does not address a protocol-version incompatibility at the DDS transport layer, since the Dashing side of the link is fixed by what physically runs on the vehicle’s ROS 2 partition and cannot itself be upgraded without departing from the vendor-supported software image.

4.2 Bridge Architecture Overview

The architecture built in response uses no ROS 2 at all on the QCar’s own compute. Two plain Python processes run directly on the Jetson TX2 (no `colcon` build, invoked as `python3 <script>.py`):

- `qcar_bridge.py` – a TCP server (port 5555) wrapping the Quanser HAL/PAL QCar object (Section 2.6): receives newline-delimited JSON drive/steering commands and responds with odometry JSON over the same full-duplex connection.
- `qcar_lidar_node.py` – a second TCP server (port 5556) wrapping the LiDAR driver, streaming scan data as newline-delimited JSON.

On the development PC, a single node, `qcar_relay_node.py`, is the sole point of contact between ROS 2 and this TCP/JSON protocol: it subscribes to `/cmd_vel` and forwards commands into the motor socket, and publishes `sensor_msgs/LaserScan` on `/scan` and `nav_msgs/Odometry` plus the `odom` → `base` TF on `/odom` from data received back over both sockets. Every other node in the navigation stack – Nav2, AMCL, Cartographer, RViz – interacts with this relay node exactly as it would with the simulation’s own drive plugin, requiring no awareness that the underlying vehicle is real hardware reached over a bespoke protocol rather than a co-located Gazebo simulation.

Listing 4.1 shows the command-receiving side of this protocol inside `qcar_bridge.py`’s main loop: incoming TCP bytes are accumulated into a buffer and split on newlines, each complete line is parsed as one JSON object, and a malformed line is logged and discarded rather than allowed to crash the control loop – deliberately fail-soft, since this loop is also driving the physical motor and must keep running even if one message is corrupt.

```

1 chunk = conn.recv(4096)
2 if chunk == b'':
3     print('dev PC relay disconnected')
4     break
5 buf += chunk
6 while b'\n' in buf:
7     line, buf = buf.split(b'\n', 1)
8     if not line.strip():
9         continue
10    try:
11        cmd = json.loads(line)
12        target_linear = float(cmd['linear_x'])
13        angular_z = float(cmd['angular_z'])
14        target_steering = compute_steering(target_linear, angular_z)
15        last_cmd_time = time.time()
16    except (ValueError, KeyError, TypeError):
17        print('bad command line, ignoring:', line)

```

Listing 4.1: Newline-delimited JSON command parsing inside `qcar_bridge.py`’s main loop.

Why TCP, why newline-delimited JSON, and why one socket per direction. Three protocol design choices underlie Listing 4.1 and are worth making explicit, since each addresses a specific failure mode rather than being an arbitrary convenience. First, **TCP rather than UDP**: TCP guarantees in-order, complete delivery of every byte sent (retransmitting silently on packet loss) at the cost of unbounded, variable latency under loss, whereas UDP offers low,

predictable latency but silently drops or reorders packets under loss with no delivery guarantee at all. For velocity and steering commands, a stale duplicate or a dropped byte silently corrupting the JSON stream is a worse failure than an occasional extra tens of milliseconds of latency (which the `CMD_TIMEOUT` watchdog in Section 4.4 already treats as a safety condition regardless of its cause), so TCP’s reliability guarantee was preferred over UDP’s lower-latency-but-lossy profile. Second, **newline-delimited JSON rather than a fixed-length or length-prefixed binary framing**: TCP is a byte stream with no built-in concept of message boundaries, so *some* framing scheme is unavoidable; a length-prefixed binary format would be marginally more compact, but a plain ASCII delimiter is trivially inspectable with a bare `nc/telnet` session during debugging (a real, exercised advantage throughout this project’s development, not a theoretical one) and requires no shared binary schema to keep in sync between the QCar-side Python and dev-PC-side Python. This is also why `buf` accumulates across `recv()` calls rather than assuming one `recv()` returns exactly one message: TCP’s byte-stream semantics make no guarantee that message boundaries align with any particular `recv()` call’s return, so a message can legitimately arrive split across two calls, or two messages can arrive concatenated in one call – Listing 4.1’s `while b'\n' in buf` loop handles both cases uniformly by always operating on whatever has accumulated so far. Third, **one full-duplex socket carrying both directions** rather than two separate one-way sockets for commands and odometry: a single TCP connection is inherently bidirectional (each endpoint can read and write independently on the same socket), so Section 4.4’s later addition of odometry sent back to the relay required no new listening port or connection – only using a direction of the existing socket that had, until then, been opened but left unread.

Both onboard TCP servers were built to handle `SIGTERM` identically to `SIGINT` – a deliberate correctness fix, not a default: Python does not automatically translate a plain `pkill`’s default `SIGTERM` into a caught, clean-exit signal the way it does for `SIGINT` (Ctrl-C), and an earlier plain `pkill` against the LiDAR process left the physical LiDAR unit spinning after the owning process had already exited, which is both a battery drain and a safety concern for anyone near the vehicle. Section 4.7 presents the resulting documented shutdown procedure in full.

4.3 LiDAR Angle-Convention Correction

The first sensor-correctness issue found while bringing up visualisation of the bridged data (`robot_state_publisher` + `joint_state_publisher` + RViz, no Gazebo) was a LiDAR angle-binning defect in `qcar_lidar_node.py`: obstacles rendered in RViz did not appear at their true bearing relative to the rendered robot model. This required *two* stacked, independent corrections, not one – a clockwise-to-counter-clockwise direction flip *and* a $+90^\circ$ rotational offset between the LiDAR’s zero-reference bearing and the vehicle’s own forward direction. The first attempted fix (direction only) produced a result that was itself the tell that a second error remained: it exactly mirrored the original error about the vehicle’s forward axis rather than correcting it, which is the expected signature of fixing only a sign error while a separate rotational offset error is still present. The final, verified-correct transform is

$$\theta_{\text{target}} = \text{wrap}_{(-\pi, \pi]} \left(-\theta_{\text{lidar}} + \frac{\pi}{2} \right), \quad (4.1)$$

verified against a real obstacle at a known bearing (the vehicle’s physical right side), cross-checked against the rendered robot model’s own orientation in RViz rather than the screen’s raw left/right, which depends on the RViz camera’s own facing direction and is therefore not a reliable reference on its own.

Listing 4.2 shows Equation 4.1 as implemented in `qcar_lidar_node.py`: each raw (`angle`, `distance`) sample from the HAL is transformed, wrapped into $(-\pi, \pi]$ via `atan2`, and binned onto the fixed 360-sample grid a ROS 2 `LaserScan` message requires – the HAL does not guarantee its raw samples already arrive pre-gridded in order, so this binning step is necessary independently of the angle correction itself.

```

1 increment = (2.0 * math.pi) / NUM_MEASUREMENTS
2 ranges = [float('inf')] * NUM_MEASUREMENTS
3 for angle, dist in zip(lidar.angles, lidar.distances):
4     target = -angle + math.pi / 2
5     normalized = math.atan2(math.sin(target), math.cos(target))
6     idx = int(round((normalized + math.pi) / increment)) % NUM_MEASUREMENTS
7     if RANGE_MIN <= dist <= RANGE_MAX:
8         ranges[idx] = float(dist)

```

Listing 4.2: LiDAR angle correction and grid binning (`qcar_lidar_node.py`).

Worked example: why a direction flip alone is not enough. Equation 4.1’s two-part correction is easiest to see by tracing a few canonical raw LiDAR angles θ_{lidar} through the transform $\theta_{\text{target}} = \text{wrap}(-\theta_{\text{lidar}} + \pi/2)$ in Table 4.1. A *direction-flip-only* correction ($\theta_{\text{target}} = -\theta_{\text{lidar}}$, no $+\pi/2$ term) would map the same four raw angles to $0, -\pi/2, \pi, \pi/2$ respectively – every value rotated exactly 90° short of the fully corrected column. This is precisely the signature Section 4.3 describes: a direction-only fix mirrors the error about the vehicle’s forward axis rather than removing it, because negating the angle alone corrects the sense of increasing angle (clockwise vs. counter-clockwise) without also correcting *where zero itself points* – two logically independent properties of an angular convention, which is exactly why this bug needed two independent corrections rather than one.

Table 4.1: Illustrative raw-to-corrected LiDAR angle mapping (Equation 4.1).

θ_{lidar} (raw)	0	$\pi/2$	π	$-\pi/2$
θ_{target} (corrected)	$\pi/2$	0	$-\pi/2$	π

4.4 Wheel Odometry over the Bridge

Mapping (Cartographer, Section 2.4) was successfully brought up on real hardware using scan-matching alone, with `use_odometry: false` / `provide_odom_frame: true` in the Cartographer configuration – meaning the initial bridge implementation required no odometry extension at all for mapping to work end-to-end. Localisation (AMCL) does not share this property: unlike Cartographer’s self-contained scan matching, AMCL looks up a live `odom` $\rightarrow$ `base` TF at every scan callback as its motion-model input, so the wheel-odometry extension originally scoped only

for the mapping phase became a hard requirement once navigation (Nav2/AMCL) was brought up.

`qcar_bridge.py` was extended with an `Odometry` class implementing the same bicycle-model dead-reckoning integration as Equations 2.1–2.3, sent back to the relay over the same already-full-duplex TCP connection the drive commands arrive on (the reverse direction of this connection had previously been opened but its data discarded). `qcar_relay_node.py` parses this stream and publishes both `/odom` and the corresponding TF. Tested in isolation before layering Nav2 on top, the resulting odometry published at approximately 45 Hz, and a 3-second commanded straight-line drive at 0.3 m/s produced $x \approx 1.25$ m, $v \approx 0.37$ m/s, $y \approx 0$ – consistent with straight-line motion at close to the commanded speed, and confirmed working before the full navigation stack was brought up on top of it.

Listing 4.3 shows the `Odometry` class in full: `update()` is called once per control-loop cycle with the raw encoder count and the current commanded steering angle, and directly implements the bicycle-model relationship of Equations 2.1–2.3 – encoder-count deltas become a travelled distance via the fixed `METERS_PER_COUNT` conversion, from which velocity and yaw rate follow, and $x/y/yaw$ are then integrated forward exactly as in the continuous-time model.

```

1 class Odometry:
2     def __init__(self):
3         self.x = 0.0
4         self.y = 0.0
5         self.yaw = 0.0
6         self.last_encoder = None
7
8     def update(self, encoder_counts, steering, dt):
9         if self.last_encoder is None:
10            self.last_encoder = encoder_counts
11            delta_counts = encoder_counts - self.last_encoder
12            self.last_encoder = encoder_counts
13
14            distance = delta_counts * METERS_PER_COUNT
15            v = distance / dt
16            yaw_rate = v * math.tan(steering) / WHEELBASE
17
18            self.yaw += yaw_rate * dt
19            self.x += distance * math.cos(self.yaw)
20            self.y += distance * math.sin(self.yaw)
21
22            return {'x': self.x, 'y': self.y, 'yaw': self.yaw,
23                    'v': v, 'yaw_rate': yaw_rate}

```

Listing 4.3: Bicycle-model dead-reckoning odometry (`qcar_bridge.py`).

From raw encoder counts to metres. `METERS_PER_COUNT` (documented in `qcar_bridge.py` as $\frac{0.01977}{2880}$) is itself a small derived constant, not a directly measured one: 2880 is the number of raw encoder counts per one full revolution of the motor shaft, as documented in the Quanser HAL

manual for this drive motor/encoder pairing, and 0.019 77 m is the linear distance one rear-wheel revolution advances the vehicle (i.e. the wheel’s rolling circumference, folding in the fixed gear ratio between motor shaft and wheel), so their ratio converts a raw encoder delta directly into a travelled distance in one multiplication, with no separate gear-ratio or wheel-circumference term needed at the call site.

The integration is forward Euler, not an exact arc. Line-by-line, `update()` performs a standard forward-Euler discretisation of Equations 2.1–2.3: it advances `self.yaw` by `yaw_rate * dt` *first*, then immediately uses that already-updated yaw to project the travelled `distance` onto x and y via $\cos(\text{yaw})/\sin(\text{yaw})$ – effectively assuming the vehicle’s heading was already at its new value for the entirety of this cycle’s straight-line displacement, rather than integrating the true curved arc the vehicle followed while turning during the interval. This is a standard, deliberate simplification (matching how Chapter 3’s simulation-side plugin also integrates its own motion incrementally rather than analytically per step), and its error shrinks toward zero as the control loop’s period `dt` shrinks, but it is not exact for any nonzero `yaw_rate * dt` – at this project’s 50 Hz control rate (Section 4.2), `dt` is small enough that the resulting error is negligible next to this thesis’s other, much larger sources of odometry inaccuracy (steering trim, throttle calibration), but it is worth naming explicitly as a non-zero, if small, contributor to dead-reckoning drift over a long enough drive.

4.5 Steering-Trim Calibration

A small but consistent steering bias – the physical vehicle drifting laterally under a nominally straight (zero-steering) command – was present on this specific chassis, and the physical steering linkage could not be adjusted to correct it mechanically. The fix was implemented in software instead, as a constant offset (`STEERING_TRIM`, an early estimate of -0.08 rad, later refined to -0.087 rad in Section 4.5.1 below) applied only at the point the steering command is written to the servo. Calibration proceeded in two stages: an initial static test (motor off, holding candidate angles and judging visually against a reference, preserved as `calibrate_steering.py` for future re-calibration) narrowed the value to roughly the right range, and a series of real straight-line driving tests (drive a known distance, measure the resulting lateral offset, compute $\delta_{\text{trim}} = \arctan(\text{offset}/\text{distance})$) refined it further. This data carried real measurement noise – lateral offset depends on the vehicle’s initial aim at the start of each test run, not purely on trim – so the calibration deliberately did not over-fit a precise value from only two data points; a consistent directional nudge toward zero observed drift converged faster in practice than attempting a precise interpolation. This first-pass value was accepted once a 4.4 m test drive showed no reported drift; it is described as “almost accurate,” a deliberate acceptance of a small residual rather than a claim of exact zero – and, as the next section shows, it was in fact only half of the real correction this vehicle needed.

4.5.1 Steering Gain: A Second, Independent Nonlinearity

The trim calibration above corrects only the steering command’s *offset* – what the vehicle does at a commanded angle of zero. It says nothing about whether a nonzero commanded angle produces

the correct amount of real wheel deflection, and for a long period of this project nobody had verified that separately. The gap surfaced directly during Nav2 integration (Chapter 6): the vehicle’s operator reported that autonomous navigation on real hardware was “not turning as well as sim,” a complaint that a pure offset error cannot explain, since an offset only shifts the zero-point and leaves proportionality untouched.

A false lead, and why it is reported here anyway. The first investigation session produced data that looked like a hard nonlinearity: a sharp left/right asymmetry (the right side matching the kinematic prediction, the left side showing roughly six times the expected deflection), and, on a later resumption, an apparent narrow “backlash snap” between two adjacent commanded angles. Both were artefacts, not steering-mechanism properties, and both were only discovered by direct physical inspection rather than further data analysis: a nut had become physically lodged under one wheel for an unknown subset of test runs, obstructing motion outright on some trials and adding partial drag on others that still nominally completed; separately, the test battery had drifted to a near-depleted 11.9 V, and the 0.3 m/s test speed used that session sat at or below the drifted static-friction floor characterised in Section 5.5.2 – one “successful” run’s own logged cruise velocity showed only ≈ 0.22 m/s actual speed for a 0.3 m/s command, against a validated 0.31 m/s at the same command from a fresh-battery session weeks earlier. Neither confound is a property of the steering system, and every conclusion drawn from that dataset was retracted rather than reconciled with the clean data below. This episode is included in full, rather than quietly omitted in favour of only the clean result, because it is the same methodological lesson Section 5.5.4 draws from a different bug: an apparently sharp, structured signal in real-hardware data is not, on its own, evidence of a matching structured cause, and a physical inspection of the test rig is sometimes a faster and more reliable diagnostic step than further statistical analysis of already-collected data.

A clean re-test. With the nut removed, a fresh battery fitted, and the test speed raised to 0.4 m/s (confirmed via logged velocity to sit safely above the stiction floor), a seven-point sweep of commanded steering angles was driven, each a fresh 1 m arc from a consistent physical start pose, with `STEERING_TRIM` held at zero so the commanded value was exactly what reached `car.write()`. The real wheel angle implied by each measured lateral deviation was back-solved from the same circular-arc geometry as the trim calibration above ($R = L / \tan \delta$, $y = R(1 - \cos(s/R))$), wheelbase $L = 0.25725$ m), using bisection rather than a closed-form inverse since the forward relationship is transcendental in δ . Table 4.2 presents the results.

The relationship is clean, monotonic, and shows no sign flips or stalls. A least-squares linear fit, $\delta_{\text{real}} = m \delta_{\text{cmd}} + c$, gives $m = 1.39$, $c = 0.119$ rad, $R^2 = 0.977$ (the two largest-magnitude commands sit slightly above the fitted line, a mild hint of additional curvature at large angles not investigated further). Two conclusions follow directly: first, the true zero-point trim is $\delta_{\text{cmd}} = -0.09$, not the -0.08 found by the coarser static test above, pinned by direct interpolation between the -0.085 (implied $+0.005$) and -0.09 (implied 0.000) rows; second, and independently of the trim value entirely, the real wheel turns approximately 39% *further* than the kinematic model’s commanded angle implies for any non-zero command – a genuine multiplicative gain error, not an offset, and one that a pure trim correction cannot fix. This gain

Table 4.2: Commanded steering angle versus implied real wheel angle (`STEERING_TRIM=0`).

Commanded angle (rad)	Implied real angle (rad)
0.00	+0.112
-0.05	+0.052
-0.08	+0.018
-0.085	+0.005
-0.09	0.000
-0.10	-0.021
-0.12	-0.062

error is a strong candidate explanation for the original complaint that motivated the investigation: Nav2’s planner and MPPI controller both assume the kinematic model of Section 2.2.1 maps 1:1 onto real wheel angle, so a vehicle that over-rotates by 39% for every commanded turn would visibly overshoot turns and cut corners tighter than planned – exactly the symptom reported.

Fix and validation. A documentation check first confirmed that no factory steering calibration exists to consult: the Quanser hardware manual documents only the raw servo command range and the physical maximum wheel angle (Table A.1), and the HAL’s own `steeringBias` construction parameter was confirmed left at its default of zero in `qcar_bridge.py` – nothing at the firmware level was fighting or duplicating the software correction. `STEERING_GAIN = 1.39` (Listing 4.4’s `apply_trim()`, which pre-divides the kinematic angle by this constant before adding `STEERING_TRIM`) was set directly from this regression. The trim itself was then refined further, from three repeated trials each at commanded -0.08 , -0.085 , and -0.09 (to average out human placement and measurement noise): the zero-crossing of the three averaged measurements gave a final `STEERING_TRIM = -0.087`. A dedicated validation drive – commanding a real kinematic turn of 0.15 rad over a 1 m arc at 0.4 m/s, predicting 28.5 cm of lateral deflection – measured 27 cm, a residual error of $\approx 5.6\%$, down from $\approx 39\%$ with no gain correction at all. The fix was subsequently confirmed in genuine closed-loop use: re-running autonomous Nav2 navigation on the physical vehicle after deployment, the operator’s direct assessment was that turning behaviour had visibly improved, consistent with the gain error having been the dominant cause of the original complaint rather than merely a contributing factor.

As with the throttle calibration constants of Section 5.5.3, both `STEERING_GAIN` and `STEERING_TRIM` are per-unit values, expected to differ on a different physical chassis; the empirical sweep-and-regression *method* used to find them is the transferable result, not the specific numbers. One operational caution from this investigation is also worth recording: testing a previously untested steering-angle magnitude can produce more real curvature than expected, so available clearance should be confirmed in the new command’s actual curve direction before testing it, not only in directions already validated at smaller magnitudes.

4.5.2 A Trim-Placement Bug and the “Apply at the Actuator Boundary” Principle

The first implementation of this trim contained a real bug worth recording as a general principle for any per-unit hardware calibration constant, not only this one. It folded the trim into the

same steering value used both for the real servo command *and* for the odometry dead-reckoning calculation in Section 4.4. Since the entire purpose of the trim is to make the real wheels read approximately zero deflection despite a nonzero servo command, feeding that same trimmed value into odometry made the dead-reckoning model believe the wheels were deflected by the trim amount even while the vehicle was, in reality, driving dead straight. A live test briefly exposed this directly: a straight-line drive of approximately 2.8 m produced a fictitious $y \approx -1.72$ m in the odometry output, a spurious $\sim 30^\circ$ curve that did not correspond to any real vehicle motion.

The fix separated `compute_steering()` (returns the untrimmed, kinematic steering angle, used for odometry) from a new `apply_trim()` (computes the real servo command, called only immediately before the HAL write), shown together in Listing 4.4. Note that `target_steering` – `compute_steering()`’s untrimmed return value – is what Listing 4.3’s `odom.update()` call is given; `apply_trim()` is invoked only once, at the `car.write()` call site itself.

```

1 def compute_steering(linear_x, angular_z):
2     # Returns the KINEMATIC (untrimmed) angle - used for odometry.
3     # STEERING_TRIM must NOT be folded in here (see prose above).
4     if abs(linear_x) < 1e-2:
5         steer = 0.0
6     else:
7         steer = math.atan(WHEELBASE * angular_z / linear_x)
8     return max(-MAX_STEER_CMD, min(MAX_STEER_CMD, steer))
9
10
11 def apply_trim(kinematic_steering):
12     # Converts a real/kinematic steering angle into the servo command
13     # that achieves it on this vehicle - call only right before
14     # car.write(), never upstream of odometry or anything else that
15     # treats its input as the true physical wheel angle.
16     return max(-MAX_STEER_CMD,
17               min(MAX_STEER_CMD,
18                   kinematic_steering / STEERING_GAIN + STEERING_TRIM))
18

```

Listing 4.4: The kinematic-steering / servo-trim split (`qcar_bridge.py`).

Re-verified with the same drive command, the odometry then correctly read $y \approx 0$. The general principle this illustrates – and which is stated here explicitly because it is expected to generalise to any future per-unit hardware calibration constant on this or a similar platform – is that a calibration constant compensating for a specific actuator’s physical imperfection should be applied *as late as possible, immediately at the actuator boundary*, and never upstream of any computation (such as odometry) that treats its input as an accurate, ground-truth kinematic quantity.

4.6 Clock Synchronisation and the Receipt-vs-Capture Timestamp Bug

The first real navigation-goal test run independently by this project’s author, after mapping, localisation, and the fixes above were all in place, surfaced a new symptom: sudden AMCL localisation jumps and visible path deviation on both straight and curved segments. The

description of what was actually observed – LiDAR scan shape stays internally coherent, but the entire scan rigidly shifts and appears to track the robot’s motion, “like the skeleton of the map is moving” – together with an explicit instinct that this might be a transform-related issue rather than a tuning problem, pointed toward a *timing* rather than a spatial or gain-tuning cause, and this proved correct.

`/scan` (port 5556) and `/odom` (port 5555) are two independent TCP streams, each subject to its own, uncorrelated network jitter – this link had already been measured showing 87 ms to 300 ms of variable latency. `qcar_relay_node.py` had been stamping both incoming streams with `self.get_clock().now()` at the moment of TCP *receipt* on the development PC, rather than at the moment of sensor *capture* on the QCar. Because the two streams’ receipt times are decorrelated, a late-arriving scan could be paired – purely by receipt-time coincidence – against an odometry sample that, in real physical time, was already stale by a different, independent amount. This produces exactly the observed symptom: a LiDAR scan that is internally self-consistent but rigidly offset relative to the map, tracking the robot’s own recent motion.

The fix required both machines’ clocks to be related, not merely each stream’s own internal timing corrected. A round-trip-corrected NTP-style offset measurement (five clustered trials of $\text{offset} = t_{\text{remote}} - (t_1 + t_2)/2$ around an SSH `date` call) found a genuine ~ 0.50 s clock offset between the two machines – a naive sequential comparison without the round-trip correction would have overstated this by also counting SSH connection-setup time as clock skew.

Why the round-trip correction is needed. A naive measurement – record a local timestamp t_1 , SSH into the QCar and read its clock to get t_{remote} , then compute $t_{\text{remote}} - t_1$ – conflates two effects that a single one-way measurement cannot separate: the genuine clock offset between the two machines, and the real, nonzero time the SSH round trip itself takes to establish a connection and execute the remote command, which inflates the apparent gap in one direction only. The correction used here follows the same symmetric-delay assumption underlying NTP itself: record a second local timestamp t_2 immediately after the remote reply arrives, and treat the remote reading as having been taken at the network’s best available estimate of the corresponding local instant, the midpoint $(t_1 + t_2)/2$ – this assumes the outbound and return legs of the round trip take approximately equal time, which is reasonable for a short, otherwise idle local-network SSH round trip but would not be for a highly asymmetric link. Averaging five such trials, rather than trusting a single one, further reduces the influence of any one trial’s round-trip time deviating unusually from that symmetric-delay assumption. This offset was deliberately *not* corrected via a system-level NTP/chrony configuration, since the QCar is shared laboratory hardware, and a persistent system configuration pointing it at one researcher’s personal development machine as a time reference is not appropriate to leave on shared infrastructure. Instead, `qcar_bridge.py` and `qcar_lidar_node.py` were changed to stamp every message with their own local `time.time()` at the moment of capture (a `'t'` field in the JSON payload), and `qcar_relay_node.py` gained a `qcar_clock_offset` parameter and a `qcar_time_to_stamp()` helper that converts a QCar-local capture timestamp into its development-PC-clock equivalent, applied consistently to both the `/odom+TF` and `/scan` publication paths, shown in Listing 4.5.

```

1 def qcar_time_to_stamp(qcar_time, qcar_clock_offset):
2     '''Converts a QCar-clock time.time() reading into a

```

```

3     builtin_interfaces/Time on this machine's clock, correcting for
4     the measured offset between the two machines' independent,
5     unsynchronized clocks.'''
6     corrected = qcar_time - qcar_clock_offset
7     sec = int(corrected)
8     nanosec = int(round((corrected - sec) * 1e9))
9     return Time(sec=sec, nanosec=nanosec)

```

Listing 4.5: Converting a QCar-local capture timestamp to the development PC's clock (qcar_relay_node.py).

Table 4.3: Diagnostic measurements underlying the clock-offset fix.

Quantity	Value
Network latency (TCP link, both ports)	87 ms to 300 ms (variable)
Measured clock offset (round-trip corrected)	≈ 0.50 s
qcar_clock_offset launch parameter (verified session)	0.50

4.7 Safe Shutdown Procedure

Because two of the four processes in this bridge run directly on physical hardware with no supervising ROS 2 lifecycle to fall back on, this project established and documented a fixed shutdown sequence, using `SIGINT` exclusively rather than a plain `pkill` (per the signal-handling fix in Section 4.2): (1) stop `qcar_relay_node.py` on the development PC; (2) stop the visualisation launch process group (RViz may itself segfault on exit, a known, harmless Qt/OpenGL quirk on this system – worth confirming it has actually exited rather than assuming); (3) stop `qcar_lidar_node.py` on the QCar; (4) stop `qcar_bridge.py` on the QCar (requires `sudo`). This project's own operational experience found it is not sufficient to trust that the sequence *was run* – on one occasion the motor bridge was still listening on its TCP port after the operator believed it had been stopped – so the documented procedure explicitly includes confirming both TCP ports (5555, 5556) are released on the QCar (`ss -tlnp`) before considering the vehicle idle.

4.8 Summary

This chapter's bridge architecture makes the physical QCar appear, from the perspective of every ROS 2 node in the navigation stack, as an ordinary `/cmd_vel`-in, `/odom//scan`-out robot – despite the underlying reality of two incompatible ROS 2 distributions connected by a bespoke TCP/JSON protocol. Getting the data flowing across that bridge to a fidelity usable by AMCL and Nav2, however, required four further, independently discovered corrections (LiDAR angle convention, wheel odometry, steering trim, and inter-machine clock synchronisation) beyond simply moving bytes across the link. Chapter 5 continues directly from here, addressing the one remaining discrepancy this chapter's fixes do not touch: the relationship between a commanded velocity and the vehicle's actual physical speed.

5 System Identification and Calibration

This chapter documents the longest and most consequential investigation of the project: a real-hardware symptom in which the vehicle, commanded to travel a fixed distance and then stop, consistently travelled substantially further – in the worst observed case, roughly three times as far – before actually coming to rest. What began as a search for a physical explanation (drivetrain coasting) ended in the discovery of a stale-data artefact in the vendor hardware abstraction layer described in Section 2.6, with the true physical residual turning out to be small once that artefact was removed. The chapter also documents a control-design attempt that was tested on physical hardware without adequate prior validation and produced a genuine, dangerous oscillation – included here in full because it is as methodologically important as the investigation’s eventual successful resolution.

5.1 Problem Statement

With the bridge of Chapter 4 operational, a simple closed-loop test was used throughout this investigation: drive the vehicle at a fixed commanded speed toward a target distance, using odometry to detect when the target is crossed, and stop. Repeated trials showed the vehicle travelling substantially further than the commanded target before settling, with the overshoot growing, not shrinking, across repeated sessions under otherwise similar conditions – an early indication that the effect was systematic and reproducible rather than simple measurement noise.

5.2 Ruling Out Simpler Explanations

Several plausible causes were tested directly and eliminated before the investigation’s eventual resolution:

- **AMCL/localisation.** The original suspect. Investigating it produced one genuine, separately valid fix – raising AMCL’s `recovery_alpha_fast/recovery_alpha_slow` from 0.0 to 0.1/0.001, since a value of 0.0 allows the particle filter to freeze permanently once it (incorrectly) converges to zero uncertainty under real sensor noise, a failure mode that a clean simulation environment does not expose. This fix was kept, but does not explain the stop-distance symptom, which persisted after it was applied.
- **Command-delivery delay.** Measured directly on the QCar’s own local clock (eliminating any network-latency contribution) at approximately 0.2s from decision to applied throttle – normal, and not large enough to explain metre-scale overshoot at the tested speeds.
- **A backlog of stale `/cmd_vel` commands.** Hypothesised as queuing behind a blocking socket `send` call in the relay’s single-threaded executor. Two increasingly careful fixes were built and deployed – a staleness timestamp check, then a full non-blocking sender-thread architecture – and both were tested directly on hardware. **Neither made any measurable difference**, ruling this out as the cause.

- **Logging/print buffering artefacts.** The timing of QCar-local debug samples during the “coast” period was a steady 20 ms to 35 ms throughout, not a rapid burst consistent with a draining backlog – ruled out.

5.3 Active Braking: A Control-Design Attempt and a Safety Incident

With the above ruled out, the leading hypothesis shifted to genuine drivetrain (motor and gearbox) rotational inertia keeping the wheels turning for a period after commanded throttle reached zero – a physically plausible mechanism, and one apparently consistent with QCar-local encoder readings that showed real velocity persisting for several seconds after a stop was commanded.

A corrective control law was implemented directly in `qcar_bridge.py`: a reverse-throttle brake pulse proportional to the residual measured velocity, intended to actively cancel the coasting motion. **This was deployed directly to physical hardware without first being traced through by hand or tested in simulation.** On its first test, it produced a genuine, dangerous oscillation: the vehicle lurched forward and backward repeatedly, reaching -0.88 m/s in reverse and still accelerating, and had to be stopped manually via an SSH-issued kill command mid-test. The root cause, in hindsight, is a textbook control failure mode: a continuous proportional reaction to *any* nonzero residual velocity, including reverse velocity, with no deadband and no explicit disarm condition, means every correction’s own overshoot looks to the controller like a fresh error requiring correction – an undamped hunting loop with no mechanism to settle.

Since both attempts were reverted, no implementation of this control law survives in the repository to quote directly; Listing 5.1 instead reproduces the comment left in its place in `qcar_bridge.py`, at the site the code once occupied – the project’s own permanent record of what was tried, why it failed, and why it must not be blindly re-attempted.

```

1 # Tried active braking (brief reverse-throttle pulse proportional to
2 # residual speed when a stop is commanded) to counter drivetrain coast -
3 # REJECTED, not just untuned. It's a proportional loop on velocity sign
4 # with no deadband and one-cycle-stale (~20ms) feedback: if the pulse
5 # overshoots past zero into real reverse motion, the next cycle reads a
6 # negative v and "corrects" with a forward push, which can itself
7 # overshoot - a genuine undamped hunting oscillation, not a bad gain
8 # value. Confirmed on hardware (twice - it was reverted once already
9 # before this comment, then re-tried and reproduced the same result):
10 # car visibly lurched forward/back repeatedly and fast, had to be
11 # killed manually. Do not re-add this without a real deadband/
12 # hysteresis or a one-shot (non-continuous) brake pulse design -
13 # retuning BRAKE_GAIN alone will not fix it.
```

Listing 5.1: Comment documenting the rejected active-braking design, left in place of the reverted code (`qcar_bridge.py`).

A minimal model of why this design cannot be stable. The instability the comment above describes can be shown formally with a deliberately simplified discrete-time model that keeps only the mechanism responsible, not the full nonlinear vehicle dynamics. Let v_k be the residual measured velocity at control cycle k , and let the brake law command a throttle proportional to

that residual with gain K : $u_k = -Kv_k$. If the vehicle’s velocity responds to one cycle’s throttle command with an effective per-cycle gain a (lumping motor response, control period, and vehicle mass into a single constant, and ignoring natural friction/drag for this minimal model), the resulting closed-loop update is

$$v_{k+1} = v_k + a u_k = v_k(1 - aK). \quad (5.1)$$

This is a first-order linear recurrence, and its behaviour is governed entirely by the magnitude of $(1 - aK)$: the residual velocity decays toward zero only if $|1 - aK| < 1$, i.e. $0 < aK < 2$; it oscillates in sign every cycle if $aK > 1$; and, critically, that oscillation *grows* in magnitude, without bound, whenever $aK > 2$, since each successive v_k is then multiplied by a factor more negative than -1 . This single inequality is the formal statement of exactly what the source comment describes qualitatively – a correction that overshoots past zero into real reverse motion, prompting a next-cycle correction in the opposite direction that itself overshoots by more – and it shows directly why *no* amount of retuning K alone guarantees safety here: a (the vehicle’s true, real per-cycle response to a given throttle command) was never independently measured before this control law was deployed, so there was no way to have verified $aK < 2$ in advance from the gain value alone. A one-shot design avoids re-arming after the first correction, but Equation 5.1’s single-step overshoot condition does not require re-arming to manifest – if the very first correction alone satisfies $aK > 2$, it already produces a reverse velocity larger in magnitude than the original residual it was meant to cancel, which is consistent with the one-shot retest reproducing the same oscillation rather than being protected by its disarm condition. This is precisely the kind of hand-traceable instability check Section 1.4’s standing rule asks for *before* hardware deployment, not after.

A revised, “one-shot” design was implemented – braking fires exactly once per drive-to-stop transition and permanently disarms the instant measured velocity crosses zero, rather than re-arming from residual drift – and **reproduced the same oscillation on retest**. Given the repeated safety failure, this control approach was abandoned entirely rather than iterated on further. Both modified files were reverted to their exact pre-investigation state, verified byte-identical to the known-good version by hash comparison, with the failure documented directly in a source-code comment so that it is not blindly re-attempted in a future session.

This incident is the clearest instance in this project of the general principle stated in Section 1.4: a reactive feedback controller acting on a live sensor reading, on a system already confirmed to have real actuator lag and inertia, should be traced through by hand for an overshoot scenario – or tested in simulation first – before it is ever sent to physical hardware. “The control theory seems sound” was not, on its own, sufficient justification here, and this project treats that as a standing rule for any future reactive control logic on this platform, not merely a lesson learned in retrospect.

5.4 Sim-versus-Real Comparison

Following a methodology suggested by the author’s academic supervisor, the identical closed-loop stop-distance test (drive N metres at a fixed speed, stop the instant odometry crosses the target)

was reproduced in both the physical robot and the Gazebo simulation of Chapter 3, using the same launch stack with only the drive target switched from simulated to real hardware.

A six-point sweep across three target distances (1.0 m, 1.5 m and 2.0 m) and three commanded speeds (0.20 m/s, 0.25 m/s and 0.30 m/s) found real hardware overshoot in the range 0.58 m to 2.12 m across two sessions roughly one hour apart, with *every* tested condition worse in the second session by a broadly consistent margin – evidence of real, reproducible drift (plausibly battery voltage or floor/tyre condition) rather than measurement noise. The matched simulation sweep, over the same six conditions, showed overshoot of only 0.044 m to 0.067 m – 10 to 30 $\times$ smaller, and only weakly sensitive to speed, in contrast to the real hardware overshoot, which scaled roughly with the cube of speed. A dedicated, isolated comparison at 2 m / 0.3 m/s sharpened this further: the simulated vehicle settled at 2.067 m (an extra 0.067 m), while the real vehicle settled at 3.91 m (an extra 1.90 m) – a $\sim 28\times$ difference for an identical software command. This result strongly supports a real-hardware-specific controller/dynamics cause rather than something inherent to the stop-command approach itself, and directly motivated the calibration work in the remainder of this chapter.

5.5 Throttle Calibration

5.5.1 The Uncalibrated Gain Hypothesis

`qcar_bridge.py`'s mapping from `cmd_vel`'s `linear.x` (in m/s) to the HAL's PWM duty cycle had, since early in the project, used a placeholder gain (`THROTTLE_GAIN = 1/3`), explicitly documented in its own source comment as uncalibrated. Direct measurement from the sweep data above confirmed this mattered: commanding 0.3 m/s produced a real, QCar-local encoder-measured cruise velocity averaging ≈ 0.8 m/s – roughly 2.6–2.7 $\times$ the commanded speed, consistent across two independent measurements. Since coasting distance from momentum scales with the square of velocity, this alone predicts roughly a 6.9 $\times$ larger coast distance than a genuine 0.3 m/s stop would produce – a substantial fraction of the sim-vs-real gap found above, and a reminder that the sim-vs-real comparison in Section 5.4 was not, at that point, comparing truly matched real speeds.

5.5.2 First Calibration Attempt: A Deadband Discovered, Then Reverted

A dedicated measurement tool, `check_stop_latency.py`, was built to measure cruise velocity and stop latency in a single run. An initial correction – lowering `THROTTLE_GAIN` by dividing by the previously measured 2.67 $\times$ ratio – was deployed to hardware and immediately surfaced a second, independent, hard non-linearity: at the old gain, a commanded 0.25 m/s ($\sim 8\%$ duty) sat completely motionless for ≈ 2.1 s before suddenly breaking static friction and moving, while 0.20 m/s and 0.15 m/s ($\sim 6.7\%/5\%$ duty) never moved at all across 8 s to 10 s of commanding. This static-friction deadband is a genuine physical property of the vehicle, not a linear gain error, and it interacts badly with a naive linear-gain correction: the lowered gain restored accurate cruise-speed tracking at higher commanded speeds, but pushed the duty cycle for normal teleoperation speeds below the deadband threshold entirely, making the vehicle stop responding to real driving commands. The gain correction was reverted the same day to restore

usable teleoperation.

This is a different friction phenomenon from Section 2.3.2's. It is worth being precise about which friction is responsible here, since this thesis discusses two physically distinct friction phenomena under similar language. Section 2.3.2's Coulomb tyre-ground friction (the simulation's μ_1/μ_2) governs whether a spinning wheel's rotation converts into forward chassis thrust at all, and is essentially instantaneous – it has no notion of a wheel being “stuck.” The deadband characterised here is a different phenomenon internal to the real drivetrain itself (motor brushes, gearbox mesh, bearing seats): *static* friction (often called stiction) resists the onset of relative motion between two surfaces up to some breakaway force threshold, while *kinetic* friction, which applies once motion has actually begun, is often measurably lower than that static threshold for the same surface pair. A commanded duty cycle below the static threshold produces zero motion no matter how long it is held, since the applied torque never exceeds the force needed to break static friction in the first place; once duty is raised just past that threshold, the *lower* kinetic friction then applies to an already-moving system, so the same duty cycle that only barely broke static friction can accelerate substantially further than expected once moving – the stick-slip signature Section 5.5.3 identifies directly in the duty-sweep data (Table 5.1's 0.085 row, which *runs away* rather than settling, despite that same duty needing no help at all breaking away from a standstill).

This reframes the calibration problem: gain and deadband are not independent. Any solution must jointly (a) track commanded speed accurately at normal operating speeds and (b) keep every one of those same commands above the static-friction threshold – which a single linear multiplier applied to the whole range cannot do, since scaling the gain down to fix (a) simultaneously worsens (b) for exactly the commands that were already only barely clearing the threshold.

5.5.3 Final Calibration: An Empirical Duty Sweep

Given that finding, the calibration approach was reframed entirely: rather than extrapolating a linear gain from one or two data points, `THROTTLE_GAIN` was forced to zero and a dedicated, lighter-weight tool (`drive_and_log.py`) was used to sweep fixed PWM duty values directly, with the vehicle on the ground under real load throughout. Table 5.1 summarises the resulting duty-response curve.

Table 5.1: Fixed-duty sweep results (QCar on the ground, real load).

Duty	Observed behaviour
0.050	No motion at all (confirmed twice, up to 27.5s of commanding)
0.060–0.061	Reliable breakaway after ≈ 2.0 s to 2.1s of stiction delay; settles to a stable, non-runaway cruise of ≈ 0.24 m/s to 0.33 m/s
0.065	≈ 0.39 m/s
0.085	Runs away to ≈ 0.85 m/s to 0.9 m/s – a stick-slip signature (static friction exceeds kinetic friction, so once moving, the same duty produces far more net accelerating force), confirmed by watching the full velocity trace climb continuously rather than plateau

A caution surfaced during this sweep and worth recording explicitly: the motion/no-motion

threshold near duty ≈ 0.06 is genuinely noisy, not a hard deterministic cut-off – a re-test with duty adjusted by only $+0.0013$ (0.0533 vs. 0.052) flipped a command from “27 s of zero motion” to reliably covering the full target distance. This floor appears to drift somewhat both within and across sessions (plausibly floor contact, resting wheel position, temperature, or battery voltage), and no single threshold measurement from this sweep should be treated as an exact, permanent constant.

The deployed calibration combines a deadband floor with a linear gain above it, fit from the duty= $0.061/0.33$ m/s and duty= $0.065/0.39$ m/s points and validated separately:

$$\begin{aligned} \text{duty} &= \text{THROTTLE_DEADBAND} + \text{THROTTLE_GAIN} \cdot |v_{\text{cmd}}|, \\ \text{THROTTLE_DEADBAND} &= 0.04, \quad \text{THROTTLE_GAIN} = 0.0667. \end{aligned} \quad (5.2)$$

Listing 5.2 shows Equation 5.2 as implemented in `qcar_bridge.py`’s control loop, together with the rate limiter that caps how fast `current_throttle` (the value actually written to the HAL) may change per cycle – a separate safeguard from the deadband/gain calibration itself, present throughout this chapter’s testing to keep duty-cycle steps within the documented 100 %/s hardware limit (Appendix A.1).

```

1 if abs(target_linear) < 1e-3:
2     desired_throttle = 0.0
3 else:
4     desired_throttle = math.copysign(
5         THROTTLE_DEADBAND + THROTTLE_GAIN * abs(target_linear), target_linear)
6     desired_throttle = max(-THROTTLE_LIMIT, min(THROTTLE_LIMIT, desired_throttle))
7     max_step = THROTTLE_RATE_LIMIT * dt
8     delta = max(-max_step, min(max_step, desired_throttle - current_throttle))
9     current_throttle += delta

```

Listing 5.2: Duty-cycle command computation and rate limiting (`qcar_bridge.py`).

Table 5.2 presents the resulting end-to-end validation across four commanded speeds, each from a dedicated 2 m drive-and-stop test with cruise velocity measured directly from `/odom`.

Table 5.2: Throttle calibration validation (Equation 5.2).

Commanded speed	Measured cruise speed	Ratio	Note
0.2 m/s	unreliable (≈ 0.24 or no motion)	–	At the deadband edge
0.3 m/s	0.31 m/s	1.05	
0.4 m/s	0.42 m/s	1.05	
0.5 m/s	0.50 m/s	1.00	

This is a substantial improvement over the original uncalibrated gain’s $2.6\text{--}2.7\times$ overshoot: commanded speeds of 0.3 m/s to 0.5 m/s now track within $\approx 5\%$. Below ≈ 0.25 m/s to 0.3 m/s, open-loop duty control against real floor/battery/temperature-dependent friction is inherently unreliable – this is treated as a hard physical limit of the open-loop approach at the tested operating conditions, not a remaining tuning gap; reliable sub-0.25 m/s creep would require

closed-loop velocity feedback from the encoder rather than a fixed duty mapping.

5.5.4 A Stray-Process False Positive

While iterating on this calibration, a separate, purely diagnostic false alarm is worth recording as a general debugging caution. A prior `check_stop_latency.py` test process, left running in the background rather than exiting automatically once its measurement completed, continued publishing `linear.x=0.0` on `/cmd_vel` at 20 Hz, silently racing against a separately launched teleoperation node publishing on the same topic. The resulting symptom – the vehicle briefly registering a throttle command and then immediately reverting to zero within one control cycle – looked exactly like a gain, deadband, or logic bug, and was initially investigated as one. ROS 2 does not warn about multiple publishers on the same topic; it simply interleaves messages, with whichever publisher’s message arrives last in a given control cycle taking effect. The actual fix was operational, not code: kill the stray process. This project subsequently adopted two standing practices as a result – checking `pgrep -af` for topic-related script names (or `ros2 topic info /cmd_vel -verbose` for publisher count) before deep-diving into a control-logic theory whenever real behaviour contradicts an issued command, and designing one-shot diagnostic nodes to exit automatically once their measurement is complete rather than spinning indefinitely.

5.6 The Hardware Abstraction-Layer Buffering Defect

Even after the throttle calibration of Section 5.5.3 made commanded and real cruise speed match closely, the original stop-distance symptom this chapter opened with – a multi-second coast after a stop command, with substantial extra distance – was still present, apparently unaffected by the gain fix. This section documents how that residual was ultimately traced to a defect in the vendor hardware abstraction layer itself, not the vehicle’s physical dynamics at all – the project’s single most significant finding, both for its magnitude and for the methodological lesson in how long an incorrect physical explanation can survive against evidence that, in hindsight, directly contradicted it.

5.6.1 An Initial, Incomplete Diagnosis

A closer look at the raw coast-phase velocity trace from one of the calibration validation runs (0.5 m/s) showed the coast was *not* a smooth decay: velocity remained essentially flat – if anything slightly *elevated* relative to the preceding cruise average – for the first ≈ 2.0 s after “stop” was commanded, and only crashed sharply to zero in the final 0.3 s to 0.4 s. This flat-plateau-then-cliff shape is inconsistent with passive drivetrain momentum/friction coasting (which predicts continuous, gradual decay starting immediately once power is cut), and instead resembles the real motor continuing to be *actively driven* for roughly two seconds after the stop command was issued. A review of the Quanser HAL documentation found no mention of any drive-motor ramp, filter, or time constant that would explain a two-second lag of this kind – only a separate, documented 0.16 s steering-servo time constant, unrelated to the drive motor. This shifted the leading hypothesis toward a command-delivery lag in the project’s own bridge software rather than physical coasting – a genuine change of theory, captured here because it turned out to still

be one step short of the actual cause. A diagnostic instrumentation pass (logging commanded velocity, the bridge’s own internally-ramped throttle value, and real encoder velocity together, all on the QCar’s local clock to eliminate network timing) was begun but interrupted by the test vehicle’s battery running out mid-session, and resumed in a later session on a charged battery.

5.6.2 Disproving Two Hypotheses, and the Argument That Found the Real Cause

The resumed diagnostic trace showed the bridge’s own internal throttle value reaching exactly zero within 52 ms of the stop command, while the encoder-reported velocity stayed elevated for roughly two more seconds – which correctly ruled out the project’s own throttle-ramping logic as the cause, but, as understood in hindsight, did *not* actually rule out a measurement or reporting defect in the encoder-velocity reading itself, since the two values are produced through entirely separate code paths. This distinction was missed on the first pass through the data.

At this point the person operating the vehicle directly and repeatedly disputed the emerging conclusion: teleoperating the QCar from a genuine standstill, the vehicle visibly starts and stops moving *instantly* in response to a command, flatly contradicting the ≈ 2 -second delay the automated test script was measuring. Two further hypotheses were tested and disproven with direct hardware data before the actual cause was found:

- **Duty-magnitude dependence** (a higher commanded speed produces more torque and a faster stiction breakaway): tested at 0.7 m/s (duty 0.087, well above the normally tested 0.052 r to 0.075 range) – the stiction delay was still 2.092 s, statistically identical to the low-speed tests. Disproven.
- **“*v* is a stale reading, not real motion.”** Disproven by the structure of the `Odometry` class’s own update code: the reported velocity *v* and the accumulated position *x* are computed from the exact same `distance` variable within the same loop iteration, so if *v* were falsely indicating motion, the accumulated position would have to freeze at the same instant – and it demonstrably did not; distance kept growing smoothly and matched the vehicle’s real, physically measured settled position.

The argument that ultimately identified the true cause was not a further measurement, but a piece of physical reasoning offered directly by the vehicle’s operator: *an encoder is a real-time sensor – it cannot have a built-in two-second lag by itself*. This was correct, and it redirected the investigation to a question the prior analysis had not actually asked: not whether the `Odometry` class was internally self-consistent (it was), but whether the value it was consuming as its own input – the raw `car.motorEncoder[0]` read – was itself stale before the `Odometry` class ever saw it. This prompted reading the vendor HAL’s own Python source directly, rather than continuing to reason about it indirectly from documentation or from the `Odometry` class’s own behaviour.

5.6.3 Root Cause: a Ring-Buffer I/O Mode

The root cause, found in the vendor HAL source (`pal/products/qcar.py`) on the QCar itself, is exactly the `readMode` distinction introduced in Section 2.6. `qcar_bridge.py` constructed its QCar object with `readMode=1` (task-based I/O), which starts a background acquisition task *at*

object construction time – before the bridge’s listening TCP socket has even opened – filling a ring buffer of `frequency×2` samples (at this project’s 50 Hz read rate, exactly 100 samples, i.e. 2.0s) with overwrite-on-overflow behaviour. Each call to `car.read()` dequeues one buffered sample. Because `car.read()` is not actually called until a relay client connects – which can take many seconds after the bridge process starts – the buffer fills and cycles during that idle gap. Once reads begin at a consumption rate matching the production rate exactly, the read stream is permanently stuck approximately 100 samples (2.0s) behind real time, because the backlog never has an opportunity to drain.

Formalising the lag: why it is exactly the buffer depth, forever. Let the background task write into the ring buffer at a fixed production index p , incrementing by one every $1/f$ seconds ($f = 50$ Hz here), and let `car.read()` calls advance a separate consumption index c , each call returning the sample at position $c \bmod B$ (buffer capacity $B = 100$) and then incrementing c . Production begins at object construction, $t = 0$; consumption begins only once a relay client connects, at $t = T_{\text{idle}}$. During $[0, T_{\text{idle}}]$, p advances alone (production with no consumption at all), so by the time consumption starts, $p = f \cdot T_{\text{idle}}$ while $c = 0$ – a gap of $f \cdot T_{\text{idle}}$ samples between them. If $T_{\text{idle}} > B/f$ (true here, since a relay connecting takes several seconds while $B/f = 2.0$ s), the overwrite-on-overflow buffer has already wrapped around at least once by T_{idle} , meaning the oldest sample still available to read is not the very first one produced, but the one produced exactly B samples before p ’s current value – so the very first `car.read()` call, at $t = T_{\text{idle}}$, necessarily returns a sample captured at $t = T_{\text{idle}} - B/f$, a lag of exactly $B/f = 2.0$ s, regardless of how much larger than B/f the idle gap T_{idle} itself was. From that point onward, because production and consumption both advance at the identical rate f (one `car.read()` per control-loop cycle, matching the background task’s own sampling rate), the gap between p and c – and therefore the lag between “now” and whatever sample is being read – stays fixed at exactly B samples indefinitely: neither index ever gains on the other, so the backlog that accumulated during the idle gap is carried forward forever rather than draining over time. This is precisely why the lag proved so exactly reproducible (always $B/f = 2.0$ s, never more and never less, regardless of how long the QCar sat idle before a relay connected) and why it never improved on its own the longer the system ran.

This single mechanism explains every property of the symptom observed across this entire chapter: the near-exact 2.0 s to 2.1 s figure (matching the buffer’s mathematically exact 2.0 s capacity), its remarkable stability across sessions, vehicle speeds, and physical conditions (a fixed buffer depth is indifferent to any of these), and its symmetric appearance on both the start side (the “stiction delay” of Section 5.5.2) and the stop side (the “coast”), since both are read through the same buffered path.

Fix: `qcar_bridge.py` now constructs the `QCar` object as shown in Listing 5.3 – immediate I/O, reading current hardware state directly with no background task or buffer involved at all.

```

1 # readMode=0 (immediate I/O) - was 1 (task-based I/O), which runs a
2 # background acquisition task into a 100-sample ring buffer
3 # (frequency*2) from the moment this object is constructed,
4 # independent of when car.read() actually starts being called. Since
5 # this object is created before the listening socket even opens, and

```

```

6 # car.read() isn't called until a relay connects, the buffer fills and
7 # starts overwriting during that gap - once reads begin at the matched
8 # 50Hz rate, they get permanently stuck ~100 samples (~2.0s) behind
9 # real-time, since the backlog never drains. Immediate I/O reads
10 # current hardware state directly, no task/buffer involved.
11 car = QCar(readMode=0, frequency=READ_RATE)

```

Listing 5.3: The one-line HAL construction change that fixed the buffering defect (`qcar_bridge.py`).

5.6.4 Validation

The full calibration test suite from Section 5.5.3 was re-run in its entirety on a freshly charged battery, with the duty formula (Equation 5.2) unchanged, to isolate the effect of this single fix. Table 5.3 presents the results.

Table 5.3: Effect of the `readMode=0` fix on stiction delay, extra distance, and stop latency.

Test	Stiction delay (before → after)	Extra distance (before → after)	Stop latency (before → after)
2 m @ 0.3 m/s	2.09 s to 0.117 s	0.520 m to 0.114 m	2.37 s to 0.598 s
2 m @ 0.4 m/s	2.14 s to 0.001 s	0.790 m to 0.114 m	2.40 s to 0.501 s
2 m @ 0.5 m/s	2.14 s to 0.116 s	0.923 m to 0.154 m	2.55 s to 0.550 s
1 m @ 0.3 m/s	2.14 s to 0.120 s	0.641 m to 0.118 m	2.44 s to 0.572 s
3 m @ 0.3 m/s	2.08 s to 0.088 s	0.583 m to 0.104 m	2.35 s to 0.509 s

Stiction delay collapses to 0.001 s to 0.12 s – an essentially instantaneous breakaway response, matching direct observation of the vehicle. Extra distance becomes small and roughly constant, ≈ 0.10 m to 0.15 m, regardless of speed or target distance, rather than scaling with cruise speed as previously found – confirming that the earlier “extra distance scales with speed” relationship was itself an artefact of a fixed-*duration* buffering delay applied to different real velocities, not a genuine vehicle property.

The small residual that remains after this fix is real, and is fully accounted for by ordinary physics rather than being a further open mystery: dividing extra distance by stop latency (an implied average coast velocity) yields 54 % to 61 % of cruise velocity in four of the five tests – consistent with an ordinary, roughly linear velocity decay to zero over the stop-latency window (a perfectly linear decay predicts exactly 50%). The one outlier (2 m @ 0.3 m/s, 108%) is attributable to a data-analysis artefact (the “cruise average” window for that specific test likely had not reached a true steady state, now that the run has much less runway once stiction delay is near-instant) rather than a physical inconsistency.

0.2 m/s remains genuinely unreliable even after this fix, sometimes failing to move the vehicle at all – this is the real, independent static-friction deadband characterised in Section 5.5.2, and is unrelated to the buffering defect resolved here.

5.7 Summary

This chapter’s investigation began with a symptom – travelling roughly three times the commanded stop distance – that had at least three superficially plausible explanations (localisation error, command-delivery delay, and physical drivetrain coasting), and ended by identifying two genuinely independent, compounding causes: an uncalibrated, non-linear throttle-to-speed mapping (Section 5.5), and a fixed-duration sensor-read buffering artefact in the vendor hardware abstraction layer (Section 5.6) that had, for a period of this project, been reasonably but incorrectly attributed to genuine vehicle physics. Neither cause was found by reasoning from documentation or first principles alone; both required direct, targeted measurement on the physical robot, and the second was found only after a direct physical argument from the vehicle’s operator prompted reading the vendor’s own source code rather than continuing to trust an internally self-consistent, but incompletely verified, software abstraction. Chapter 6 builds on this chapter’s calibrated, buffering-free bridge to bring up and validate full closed-loop Nav2 navigation on the physical vehicle.

6 Navigation Integration and Validation

This chapter describes bringing the full Nav2 navigation stack up against the physical QCar, building on the bridge of Chapter 4 and the calibration of Chapter 5. Section 6.1 covers porting the simulation-tuned Nav2 configuration to hardware. Section 6.2 describes a custom MPPI critic plugin developed for this vehicle. Section 6.3 reports the phased bring-up used to de-risk the integration. Section 6.5 presents a detailed timing investigation into an apparent localisation-display artefact, and Section 6.6 reports the resulting end-to-end validation.

6.1 Porting the Nav2 Configuration to Hardware

The Nav2 parameter set developed and tuned in simulation (Chapter 3) required a small, mechanical set of changes to run against physical hardware rather than Gazebo: every occurrence of `use_sim_time` (13 in total, across the costmap, planner, controller, and AMCL parameter blocks) was switched from `true` to `false`, since the physical robot has no simulation clock to synchronise against. Both mapping and navigation launch files were given a real-hardware branch, gated on `launch_sim`, alongside the existing Gazebo branch, with real hardware as the default and simulation opt-in (`launch_sim:=true`) – a deliberate inversion of which mode is the default, made once hardware became the project’s active development target, while keeping the simulation path fully intact for future regression testing rather than removing it.

The AMCL `recovery_alpha_fast/recovery_alpha_slow` fix from Section 5.2 was carried forward proactively into the hardware configuration at this point, rather than left to be independently rediscovered against hardware, since the underlying cause – Section 2.5.2’s augmented-MCL random-particle-injection mechanism being permanently disabled at $\alpha_{\text{slow}} = \alpha_{\text{fast}} = 0$, leaving the filter with no way to recover from an overconfident convergence once real sensor noise (which a clean simulation rarely exercises) triggers one – is a property of AMCL’s own resampling algorithm, not specific to any one deployment target.

6.2 Custom MPPI Critic: `EarlyCommitCritic`

Early navigation testing showed a specific weakness in the stock MPPI critic set (Section 2.5) for this vehicle: the standard path-following critics evaluate a sampled trajectory’s alignment and progress against the path as a whole, which can leave the controller driving further into an approaching curve before it begins reacting to it, rather than beginning the turn as early as physically sensible – a behaviour more forgiving for a differential-drive robot’s tight turning radius than for an Ackermann-steered vehicle with the finite minimum turning radius of Table 2.2.

A custom MPPI critic plugin, `EarlyCommitCritic`, was developed and built into the navigation package to address this: it scores only the first `early_time_steps` of each sampled trajectory rollout against the bearing to a near-term path point, with no distance or angle gating, so that MPPI is pushed to begin turning toward the path immediately rather than waiting until deeper into the curve. It is gated by an `active_path_points` parameter so that it stops influencing the controller once the vehicle has made genuine progress along the route, avoiding

interference with the stock critics’ normal path-tracking behaviour during straight segments. A later direction-awareness refinement extended the critic to handle reversing and K-turn path segments correctly, where a naive “always turn toward the next point” bearing calculation would otherwise fight the vehicle’s intended direction of travel.

Building this plugin required matching `xtensor/xsimd` compile definitions exactly against Nav2’s own MPPI controller build – a mismatch here does not produce a compile-time error, but a silent ABI incompatibility that crashes the navigation container on the first control cycle that touches vectorised critic math, making it a notably unforgiving class of build-configuration bug to diagnose from its symptom alone.

Listing 6.1 shows the critic’s `score()` method in full. It computes the bearing from each sampled trajectory’s own starting position to a near-term path target (`offset_from_furthest` points ahead of the furthest point the robot has already reached), then, for only the first `early_time_steps` of that trajectory, accumulates the angular distance between the trajectory’s actual yaw and that bearing – scoring every sampled rollout independently, not just the robot’s single current pose. The early return on `active_path_points` is what confines this to the start of a path, and the `forward_preference`-gated minimum of the forward- and reverse-bearing angular distances is the direction-awareness refinement mentioned above, letting a trajectory be scored against whichever of the bearing or its reverse it is actually closer to, rather than penalising a legitimate reversing manoeuvre.

```

1 void EarlyCommitCritic::score(CriticData & data)
2 {
3     if (!enabled_ || data.path.x.shape(0) < 2) { return; }
4     utils::setPathFurthestPointIfNotSet(data);
5
6     // Only relevant at the start of a path - once the robot has made
7     // real progress, let the normal path/goal critics take over.
8     if (*data.furthest_reached_path_point >= active_path_points_) { return; }
9
10    const size_t path_size = data.path.x.shape(0) - 1;
11    const size_t offset_idx = std::min(
12        *data.furthest_reached_path_point + offset_from_furthest_, path_size);
13    const float target_x = data.path.x(offset_idx);
14    const float target_y = data.path.y(offset_idx);
15
16    const size_t time_steps = data.trajectories.x.shape(1);
17    const size_t early_steps = std::min(early_time_steps_, time_steps);
18    if (early_steps == 0) { return; }
19
20    const xt::xtensor<float, 1> x0 = xt::view(data.trajectories.x, xt::all(), 0);
21    const xt::xtensor<float, 1> y0 = xt::view(data.trajectories.y, xt::all(), 0);
22    const xt::xtensor<float, 1> bearing = xt::atan2(target_y - y0, target_x - x0);
23    const xt::xtensor<float, 1> reverse_bearing = bearing + static_cast<float>(M_PI);
24
25    xt::xtensor<float, 1> sum_diff = xt::zeros<float>({data.trajectories.x.shape(0)});
26    for (size_t t = 0; t < early_steps; ++t) {

```

```

27  const xt::xtensor<float, 1> yaw_t = xt::view(data.trajectories.yaws, xt::all(), t);
28  const xt::xtensor<float, 1> fwd_diff =
29      xt::abs(utis::shortest_angular_distance(yaw_t, bearing));
30  if (forward_preference_) {
31      sum_diff += fwd_diff;
32  } else {
33      const xt::xtensor<float, 1> rev_diff =
34          xt::abs(utis::shortest_angular_distance(yaw_t, reverse_bearing));
35      sum_diff += xt::minimum(fwd_diff, rev_diff);
36  }
37  }
38  const xt::xtensor<float, 1> cost = sum_diff / static_cast<float>(early_steps);
39  data.costs += xt::pow(cost * weight_, power_);
40  }

```

Listing 6.1: EarlyCommitCritic::score() (src/critics/early_commit_critic.cpp).

How this cost actually changes which trajectory wins. Connecting this directly to Section 2.5.1’s softmax weighting (Equation 2.9): `data.costs` is the same per-trajectory accumulator every enabled critic adds into, so this critic’s contribution $(\overline{\Delta\theta} \cdot \text{weight_})^{\text{power_}}$ (the early-steps-averaged angular difference $\overline{\Delta\theta}$ from the listing’s `cost` variable, scaled and exponentiated by its own configured weight and power) is simply summed alongside the stock critics’ own contributions into $S(\tau^{(k)})$ before Equation 2.9’s exponential weighting is applied. Concretely: of two sampled trajectories approaching the same curve, one that begins turning toward the near-term path point immediately accrues a small $\overline{\Delta\theta}$ over its first `early_time_steps`, while one that continues driving straight before reacting accrues a much larger one – inflating that second trajectory’s total cost $S(\tau^{(k)})$, which Equation 2.9 translates into an *exponentially* smaller importance weight $w^{(k)}$ relative to the early-turning trajectory. Because the executed control is the weight-averaged sum over all K sampled trajectories, systematically suppressing the weight of every late-turning sample – rather than merely marking one specific trajectory as disqualified – is what actually shifts the controller’s averaged output toward beginning the turn earlier, without ever needing to identify or select a single “best” trajectory explicitly.

6.3 Phased Hardware Integration

Given the number of independent subsystems involved (TCP bridge, odometry, SLAM, localisation, planning, control), integration proceeded in three deliberately staged phases rather than attempting the full stack at once, so that a failure at any stage could be attributed to a bounded set of newly-added components rather than the entire system:

Phase 1 – Teleoperation. Bridge (Chapter 4) plus manual `/cmd_vel` teleoperation only, with no odometry, mapping, or navigation involved. Confirmed the motor and steering command path worked correctly end-to-end before any more complex subsystem was layered on top.

Phase 2 – SLAM. Cartographer mapping (Section 2.4) added on top of Phase 1, confirmed to require no odometry extension at all given the `use_odometry: false` configuration de-

scribed in Section 2.4. A real, growing occupancy grid was built by driving the vehicle around a bounded space under teleoperation and saved via `nav2_map_server`’s `map_saver_cli` – this saved map is the one used for all subsequent localisation and navigation testing.

Phase 3 – Navigation. The full Nav2 stack (AMCL, planning, MPPI control) added on top of Phase 2’s saved map, requiring the wheel-odometry extension of Section 4.4 (deferred from Phase 2, where it had not been necessary) and the steering-trim (Section 4.5) and throttle calibration (Section 5.5) work, since dead-reckoning odometry and AMCL localisation are only as accurate as the vehicle model producing them.

The full stack’s initial bring-up – motor bridge, LiDAR node, relay node, and the ported Nav2 launch configuration – came up cleanly on the first attempt against the saved Phase 2 map: both Nav2 lifecycle managers reported their managed nodes active, all seven configured MPPI critics (the stock six plus `EarlyCommitCritic`) loaded matching the intended configuration, and the costmap auto-sized correctly to the saved map’s dimensions, with no crash loop. A real navigation goal was deliberately not sent during this first bring-up, since dead-reckoning odometry – and therefore everything built on top of it, including AMCL and planning – is only as trustworthy as the still-unresolved steering-trim calibration at that point in the project’s timeline; that calibration (Section 4.5) was completed before the first real goal was attempted.

6.4 A K-Turn Freeze at Reversal Cusps: Diagnosis and a Custom Smoother

Beyond the display-latency and calibration issues of the preceding sections, real-hardware testing in tight spaces surfaced a distinct and more serious navigation problem: a goal requiring the vehicle to turn around produced, in the operator’s words, “a chaotic curve, then reverse with opposite curve, then forward curve” – a visibly hesitant, multi-second freeze at the direction change – rather than a clean, continuous K-turn. This section documents the diagnosis, several fix attempts that failed instructively, and a custom smoother plugin that resolved it, validated in both simulation and on the physical vehicle.

6.4.1 Reproduction

To investigate safely and repeatably, a narrow dead-end corridor (approximately 2.4m wide, 2m deep) was added to the simulation map by re-mapping the Gazebo world of Chapter 3 and saving a fresh occupancy grid via `map_saver_cli`, then sending a `NavigateToPose` goal deep inside it with a reversed final heading – a target that forces a K-turn in a space too tight to complete it in one continuous arc. Two logging tools were built for this investigation: one recording every `/plan` message (not only the first, so that every replan issued during a recovery cycle gets its own sequence-numbered trace), `/odom`, and `/cmd_vel` to timestamped CSVs, and a companion analysis script computing total manoeuvre duration, position-stagnation windows (freeze detection), and true steering-direction-reversal counts directly from those CSVs.

6.4.2 Diagnosis: The Planner Is Correct, the Controller Is Not

Reconstructing the vehicle’s true chassis heading through the manoeuvre (motion direction minus 180° after the cusp, verified against a independent bicycle-model simulation of a three-point turn)

confirmed that `SmacPlannerHybrid` was producing a kinematically correct global plan: heading rotates smoothly and monotonically through the single Reeds–Shepp cusp the plan contains, exactly the “flip steering side to continue rotating the same way” geometry a real K-turn requires. The freeze is not a planning defect at all – it is entirely in *execution*.

The root cause lies in `nav2_mppi_controller`’s path-handling logic (`path_handler.cpp`, Humble branch): MPPI is only ever shown the global plan truncated up to the cusp pose until a function called `isWithinInversionTolerances()` reports that the vehicle has reached that pose within *both* a position tolerance and a heading tolerance simultaneously – until then, none of the path beyond the cusp exists as far as the controller’s own path-following critics are concerned, so nothing in the cost function encourages progress through it. This is structurally the same failure mode as a final-goal reorientation freeze found and fixed earlier in this project’s tuning history, outside the scope of this thesis’s own narrative: a goal requiring a large final heading change would stall in the same way, for the same reason, and was resolved simply by raising `GoalAngleCritic`’s cost weight (from 3.0 to 10.0) so the controller is pushed to complete the final heading precisely. No equivalent encouragement exists for an *intermediate* cusp pose, which is exactly where this freeze occurs.

6.4.3 Three Plausible Fixes, All Reverted

Three direct attempts to relax this gating condition were each tested against the reproduction and reverted, all pointing to the same underlying lesson:

- **Widening `isWithinInversionTolerances()`’s own position and heading tolerances** (from 0.2 m/0.4 rad to 0.4 m/0.7 rad) made things worse: the controller now reported the cusp “reached” too loosely, and the resulting premature reveal of the post-cusp path triggered six or more `progress_checker` aborts before the vehicle had even physically reached the cusp.
- **Faster `stuck-detection`** (lowering `progress_checker`’s movement-time allowance from 30 s to 8 s) fired aborts every 8 s to 13 s – faster than the manoeuvre could genuinely complete – and once triggered a `BackUp` recovery action that itself failed outright for lack of room in the corridor.
- **A first-version `path-smoother`** that re-projected the path onto a straight line for a fixed distance after each cusp, then jumped instantly back to the original curve, performed worse still (never completing within a 108 s window): the plugin’s own logic worked exactly as designed, but the hard discontinuity it introduced at the blend boundary was itself a new obstacle for the controller.

All three attempts shared a common thread that became the guiding principle for the eventual fix: *the freeze needs real, uninterrupted time and space to resolve on its own; interrupting it, however that interruption is triggered, consistently made the manoeuvre worse, never better.*

6.4.4 A Reverse-Only, Gradually Blended Smoother

A second design, `CuspStraighterSmoother`, was built as a `nav2_core::Smoother` plugin – path post-processing rather than a controller-side change – with two deliberate constraints: it acts *only* on cusps the path enters in reverse (a front-steered vehicle’s steered axle is self-correcting driving forward but trailing, and jackknife-prone, in reverse, making a reversing cusp the harder sub-case; forward-entering cusps and all ordinary forward segments are left completely untouched), and it blends *gradually* rather than cutting over instantly, tapering linearly from the straightened projection at the cusp itself back to the original planned curve over a configurable `straight_distance`.

A mutation-order bug, and the fix. Directly testing this design against harder, multi-cusp replans (plausible whenever a path is replanned from a robot pose already mid-manoeuvre, e.g. during a recovery cycle) surfaced a genuine implementation bug: the original `smooth()` detected every cusp once up front, then processed each one *in place* on the very path array it was simultaneously rewriting. When two cusps fell close together, an earlier cusp’s blend could overwrite position and heading data that a nearby later cusp still needed to read in order to classify itself and compute its own straight-line target – a self-inflicted data race within a single, single-threaded function. The fix, shown in Listing 6.2, snapshots the incoming path into a read-only `original` vector before any modification begins; every geometric decision the function makes – cusp detection, forward/reverse classification, and every blend target – reads only from this untouched snapshot, and writes go exclusively to a separate output path being built alongside it.

```
1  const std::vector<geometry_msgs::msg::PoseStamped> original = path.poses;
2
3  std::vector<size_t> cusp_indices;
4  for (size_t i = 1; i + 1 < n; ++i) {
5      const double d1x = original[i].pose.position.x - original[i - 1].pose.position.x;
6      const double d1y = original[i].pose.position.y - original[i - 1].pose.position.y;
7      const double d2x = original[i + 1].pose.position.x - original[i].pose.position.x;
8      const double d2y = original[i + 1].pose.position.y - original[i].pose.position.y;
9      const double len1 = std::hypot(d1x, d1y);
10     const double len2 = std::hypot(d2x, d2y);
11     if (len1 < 1e-6 || len2 < 1e-6) { continue; }
12     const double dot = (d1x * d2x + d1y * d2y) / (len1 * len2);
13     if (dot < 0.0) { cusp_indices.push_back(i); } // direction reverses by > 90 deg
14 }
15 // Classify each cusp (forward- vs reverse-entering) from 'original' only,
16 // independent of anything the output-building loop below has written so far.
17 for (const size_t cusp_i : cusp_indices) {
18     const double cusp_yaw = tf2::getYaw(original[cusp_i].pose.orientation);
19     const double dx = original[cusp_i + 1].pose.position.x - original[cusp_i].pose.
        position.x;
20     const double dy = original[cusp_i + 1].pose.position.y - original[cusp_i].pose.
        position.y;
21     const double along = dx * std::cos(cusp_yaw) + dy * std::sin(cusp_yaw);
```

```

22   cusps.push_back({cusp_i, along < 0.0, along}); // along < 0 => reverse-entering
23 }

```

Listing 6.2: Cusp detection and reverse/forward classification, reading only from a read-only snapshot (cusp_straightener_smoother.cpp).

Directly re-tested afterward against a genuine multi-cusp replan (a 47-point path with three cusps, two of them at adjacent indices), the fixed version classified and handled both correctly – exactly the scenario the original bug would have corrupted.

Table 6.1 compares three trials each of the untouched baseline, the buggy v2 smoother, and the fixed v2 smoother, on the identical corridor reproduction goal with a fresh stack relaunch before every trial.

Table 6.1: K-turn manoeuvre duration and recovery count, corridor reproduction goal (simulation).

Configuration	Trial durations (recoveries)	Mean	Worst case
Baseline (no smoother)	81.3s(1), 81.1s(1), 168.1s(4)	110.2s / 2.0	168.1s
v2 smoother (buggy)	96.6s(2), 101.4s(2), 432.8s(12)	210.3s / 5.3	432.8s
v2 smoother (fixed)	59.3s(0), 97.1s(2), 103.7s(2)	86.7s / 1.3	103.7s

Run-to-run variance is genuinely high in every configuration – consistent with the freeze being a stochastic MPPI sampling effect rather than a deterministic planning defect – but the fixed smoother clearly outperforms the untouched baseline on every measure, while the buggy version is markedly worse than doing nothing at all, underlining how much the mutation-order bug alone was costing.

6.4.5 “Extending the Corner”: A Further, Larger Improvement

A further refinement addressed the residual freeze directly at its source: rather than requiring the path’s incoming and outgoing legs to meet at one precise geometric vertex – which `isWithinInversionTolerances()` must still converge on exactly, even with the gradual blend of Section 6.4.4 in place – a new `extend_distance` parameter pushes the cusp point itself further along the incoming leg’s own heading before the reverse blend begins, so the incoming and outgoing legs deliberately overlap rather than meeting at a single tight point, giving the controller geometric margin around the transition instead of an exact target. Listing 6.3 shows the construction: the extension point is inserted into the output path, and the existing distance-based blend is re-anchored to start from it rather than from the original cusp position.

```

1  const double cusp_yaw = tf2::getYaw(original[i].pose.orientation);
2  const double ex = original[i].pose.position.x + std::cos(cusp_yaw) * extend_distance_;
3  const double ey = original[i].pose.position.y + std::sin(cusp_yaw) * extend_distance_;
4
5  geometry_msgs::msg::PoseStamped ext_pose = original[i];
6  ext_pose.pose.position.x = ex;
7  ext_pose.pose.position.y = ey;
8  output.push_back(ext_pose);
9  // The existing straight_distance_blend (unchanged logic) now runs from
10 // (ex, ey) as its origin, not from the original cusp position - the reverse

```

```

11 // leg must travel back past this pushed-out point to rejoin the downstream
12 // curve, giving the controller a margin instead of one exact vertex.

```

Listing 6.3: Pushing the cusp outward before blending, so incoming/outgoing legs overlap (cusp_straightener_smoother.cpp).

With `extend_distance` set to 0.15 m alongside the existing 0.21 m `straight_distance`, three further trials produced 45.2 s, 38.3 s, and 48.1 s – a mean of 43.9 s with **zero recoveries in every trial**, roughly halving the already-improved fixed-v2 mean of 86.7 s on top of its improvement over the 110.2 s baseline. This is the final, locked-in configuration.

6.4.6 Real-Hardware Validation

The fixed, corner-extending smoother was tested on the physical QCar with two goals sent manually by the operator, each requiring a genuine reversal manoeuvre: a 5.15 m traverse completed in 50.1 s with zero recoveries across seven replans (cusp counts 1, 1, 1, 1, 8, 8, 0), and a second, longer traverse completed in 96.8 s, also with zero recoveries, across eight replans (cusp counts 5, 5, 5, 5, 9, 10, 5, 3). Both goals succeeded outright – the strongest available confirmation that the fix generalises beyond the simulated corridor reproduction it was developed against.

Both goals also produced noticeably higher cusp counts on individual replans (up to 8–10) than any simulation trial had shown (maximum approximately 3) – an open, unexplained gap between simulated and real replanning behaviour, plausibly `SmacPlannerHybrid` responding to more awkward real goal/environment geometry than the simulated corridor exercised, but not confirmed. It did not prevent either goal from succeeding, so it is recorded here as a genuine open item for future work (Section 8.2) rather than a present limitation of the fix.

One apparent timing anomaly in this trace was investigated and correctly ruled out rather than mistaken for a bug: the first several `smooth()` calls on the first goal landed within roughly one second of each other, which initially looked like an unthrottled replan burst. Re-reading the behaviour tree configuration showed this is by design: `IsPathValid` is already rate-limited to 1 Hz by an existing `RateController`, and a `RateController` fires immediately on its first tick before settling into its steady interval – exactly the pattern observed, once the also-unconditional, once-only `InitialComputePathToPose` call is accounted for separately. Catching this before implementing a second, redundant rate limiter avoided a change that would have had no effect at all.

6.4.7 Continued Iteration: A Re-Extension Bug and a Curvature-Continuing Refinement

Hardware testing continued beyond the validation above, and surfaced a further, genuine bug in the corner-extension mechanism itself, plus a refinement to it – both developed and sim-tested after the hardware-validated result of Section 6.4.6, and not yet re-confirmed on the physical vehicle at the time of writing; they are reported here with that status made explicit, consistent with this thesis’s practice of not presenting a provisional result as final.

Bug: the extension re-applies to the same cusp on every replan. Because Nav2 replans roughly once per second while a goal is active, and each replan’s path starts fresh from the robot’s

current pose, the *same* physical K-turn cusp reappears near the front of the replanned path on every single replan as the robot approaches and then executes it – and, since the smoother has no notion of “already handled this one,” it re-applied the full extend-and-straighten treatment to that same cusp every time. Direct log analysis of a real hardware run confirmed this concretely: one K-turn’s cusp received the treatment on eight separate replans as the robot closed in on it, its index in the (shrinking) replanned path falling from 94 down to 1 – by the final replan, the robot was already essentially at the cusp, mid-manoeuvre, and the smoother was still reshaping the path directly underneath it rather than leaving an already-committed manoeuvre alone. This is, functionally, a milder recurrence of the same “interrupting an in-progress manoeuvre makes it worse” pattern identified repeatedly elsewhere in this investigation (Section 6.4.3), this time self-inflicted by the smoother rather than by an external replan trigger.

The fix adds a `min_lead_distance` parameter (0.3m): the arc length from the replanned path’s start to each candidate cusp is computed, and the extend-and-straighten treatment is only applied if the cusp is still at least this far ahead of the robot’s current position, leaving an already-reached or already-underway cusp untouched rather than reshaping it again. Two sim trials on the standard K-turn reproduction goal after this fix showed no regression, though the specific skip condition was not directly observed firing in either trial (both completed before any replan’s cusp closed to within the threshold) – correct by code review and by the logged parameter value at start-up, but not yet directly confirmed exercising its skip branch. This is recorded as an open verification item, not a claimed result.

Refinement: continuing the incoming curve rather than cutting straight. Separately, the extension itself was upgraded from a straight-line push-out (Listing 6.3, hardware-validated above) to one that continues the incoming leg’s own curvature. The motivation is direct: cutting straight right at the corner, even when the vehicle is still visibly curving on its approach to the cusp, asks the vehicle to straighten abruptly at exactly the point it should be easing into the manoeuvre most naturally. The refined version estimates the incoming leg’s curvature at the cusp from its own recent yaw change, $\kappa = \Delta\theta/\Delta s$ over the last incoming segment, and generates the extension as a short constant-curvature arc with that same curvature rather than a straight line – built from several closely spaced sub-poses (0.03m steps) so the curve is genuinely present in the path’s own geometry, not merely implied by a single endpoint. It degrades automatically to the original straight-line behaviour when the incoming leg is already straight ($\kappa \approx 0$), so it is a strict generalisation of the hardware-validated mechanism rather than a replacement with different behaviour on straight approaches.

```

1 // kappa estimated from the incoming leg's own yaw change (dyaw/ds) at the cusp - 0 if
  the
2 // incoming leg was already straight, giving the original straight-line behaviour
  unchanged.
3 constexpr double kExtensionStep = 0.03;
4 const int extension_steps = std::max(1, static_cast<int>(std::round(extend_distance_ /
  kExtensionStep)));
5 const double step = extend_distance_ / extension_steps;
6 double ex = cx, ey = cy, ext_yaw = cusp_yaw;
7 for (int s = 1; s <= extension_steps; ++s) {

```



```

8   const double arc = step * s;
9   double px, py;
10  if (std::abs(kappa) > 1e-6) {
11      const double radius = 1.0 / kappa;
12      px = cx + radius * (std::sin(cusp_yaw + kappa * arc) - std::sin(cusp_yaw));
13      py = cy - radius * (std::cos(cusp_yaw + kappa * arc) - std::cos(cusp_yaw));
14  } else {
15      px = cx + std::cos(cusp_yaw) * arc; // kappa ~ 0: degrades to the straight-line
        case
16      py = cy + std::sin(cusp_yaw) * arc;
17  }
18  ext_yaw = cusp_yaw + kappa * arc;
19  // ... each (px, py, ext_yaw) is appended as its own pose, making the arc explicit.
20  }

```

Listing 6.4: Continuing the incoming leg’s own curvature into the extension, rather than cutting straight (cusp_straightener_smoother.cpp).

A single sim smoke test on the standard reproduction goal completed successfully (≈ 99 s, within the range of prior trials) with the new curvature-extension code path exercised across eight reverse-cusp events over the run’s several replans, with no numerical failure. This is reported as a working, sim-verified refinement, not yet subjected to the same multi-trial statistical comparison or hardware re-validation as the straight-line version it replaces – a concrete, already-identified next step (Section 8.2).

6.4.8 A Related Planner-Side Finding: Near-Wall Goals and a Tolerance Trade-off

A separate, planner-level finding surfaced during the same period of continued tuning, also directly relevant to operating in tight, wall-adjacent spaces: a goal placed only ≈ 0.20 m from the nearest mapped wall – inside the footprint clearance the vehicle’s own $0.44\text{ m} \times 0.18\text{ m}$ collision footprint requires – failed outright on every one of six recovery retries, each attempt logging `SmacPlannerHybrid` exhausting its entire search iteration budget without ever finding a collision-free plan. This is a goal-placement issue rather than a code defect: no footprint- or inflation-related configuration was implicated, and the practical guidance that follows is simply to keep any commanded goal at least 0.3 m to 0.4 m clear of a mapped obstacle, given this vehicle’s footprint.

A tempting-looking fix was tried and deliberately reverted, and the reason why is itself a useful finding. `SmacPlannerHybrid`’s own search loop, confirmed directly against its source, backtracks to the closest node it reached within a configurable `tolerance` radius rather than failing outright whenever the exact goal proves infeasible – so widening `tolerance` (from 0.5 m to 1.0 m) did immediately rescue the failing near-wall goal in a live test. However, it introduced a worse, silent failure mode in its place: the substituted pose the planner backtracks to, not the originally requested goal, becomes the plan’s actual endpoint, and nothing downstream in the standard `NavigateToPose` behaviour tree ever rechecks the final pose against the original request – so *any* marginally infeasible goal, not only the intended near-wall case, could now report `Goal succeeded` while the vehicle sat up to a full 1.0 m away from, and potentially facing the wrong

direction relative to, where it was actually asked to go, with no indication in the result that a substitution had occurred at all. An honest, visible failure is preferable to a silently mis-reported success, particularly for a system whose results this thesis reports as measured fact – `tolerance` was reverted to its original 0.5 m, accepting that the specific near-wall goal will fail outright again, since the correct fix for a genuine target at that location is to ensure real clearance there, not to relax what counts as “success” for every goal system-wide. This mirrors, at the planner level, the same safety-over-convenience judgement Section 5.3 applies at the control level: a fix that resolves the reported symptom by quietly weakening a correctness guarantee elsewhere is not an acceptable trade, even under time pressure.

6.4.9 Two Follow-On Attempts, Reverted

Two further improvements were implemented and hardware-tested the same day, and both were reverted – included here because a reverted attempt with a documented, evidence-based reason is exactly the kind of result this thesis’s methodology (Section 1.4) treats as worth reporting, not discarding.

`IsPathValidDebouncedCondition`, a custom behaviour-tree condition requiring two consecutive failed validity checks (rather than stock `IsPathValid`’s single check) before triggering a replan, improved simulation trials further still (38.2 s, 38.3 s and 40.4 s, mean 39.0 s, zero recoveries) by absorbing occasional single-tick sensor or transform noise that would otherwise trigger an unnecessary replan. On real hardware, however, a subsequent run on a freshly scanned map showed the vehicle never executing its reverse segment at all: the local costmap reported the path invalid on *every* check, not intermittently, so the debounced condition correctly (and unhelpfully) confirmed a persistent rather than transient invalidity every 1.5 s to 2.6 s, aborting the in-progress manoeuvre each time – the same “interrupting a manoeuvre makes it worse” lesson from Section 6.4.3, under a new trigger. The most likely root cause, not fully confirmed before the revert, is an unmeasured `qcar_clock_offset` (Section 4.6) on the relay for that session – a fresh measurement immediately afterward found a real ≈ 0.15 s offset. A companion `max_cusps` rejection cap was reverted alongside it to return to one single known-good state rather than debug two independent changes under time pressure, though it was not itself implicated (every replan in the failing run came back at exactly two cusps).

A custom `PathDeviationCritic` – pulling sampled trajectories toward a point further along the path once the vehicle drifts more than 0.3 m from it, rather than toward the nearest path point – was tested for three trials (112.8 s, 71.7 s and 38.4 s, mean 74.3 s, zero recoveries) against the corner-extending smoother’s own 43.9 s mean, and reverted: both the mean and the run-to-run variance were worse, plausibly because the new critic’s pull competed directly against the stock `PathAlignCritic` rather than complementing it.

6.4.10 A Related Operational Caution: Goal Preemption

A separate mechanics finding from this investigation, worth recording as an operational rule rather than a fix: if a goal is deep into a failing/retrying cycle (for example, the planner repeatedly failing to find a plan, with the behaviour tree’s recovery retries nearly exhausted) and a new goal is sent while the old one is still active, the new goal does not reliably receive a fresh attempt –

it was observed to fail almost instantly, with no real planning attempt of its own, rather than superseding the old goal cleanly. The practical rule is to let a stuck goal finish (fail out) or explicitly cancel it before sending a correction, rather than assuming a new goal always starts with a clean slate.

6.5 Diagnosing an Apparent Display-Localisation Artefact

After the throttle calibration of Section 5.5.3 (but before the buffering-defect fix of Section 5.6), a live Nav2 goal test surfaced a new symptom, described by the vehicle’s operator in detail: when driving toward a goal inside an enclosed space, the entire mapped rectangle appeared to shift in RViz, with the LiDAR scan keeping its internal shape but dragging behind the robot’s true motion before “snapping” back into alignment – and the same shift-then-correct pattern recurred, after a delay, once the goal was reported reached.

6.5.1 Ruling Out Simpler Causes

Several candidate causes were checked directly and eliminated: `qcar_clock_offset` was re-measured fresh and re-applied (with one procedural caution found along the way – a relay launch command given during this session had omitted the `qcar_clock_offset` argument entirely, silently defaulting it to zero and briefly reintroducing exactly the bug fixed in Section 4.6, worth checking for explicitly before suspecting the offset value itself); `map` → `odom` TF was captured directly during active driving and updated in small, smooth, incremental steps rather than sudden jumps; the RViz **Fixed Frame** configuration was confirmed correct (`map`); `base` → `lidar` TF is static and therefore not a timing bottleneck; local CPU load was checked and found unremarkable (RViz $\approx 10\%$, the Nav2 container $\approx 29\%$); network round-trip latency to the QCar was 1 ms to 6 ms, far too small to explain a multi-second visible effect; and direct receipt-versus-capture latency on `/odom` and `/scan` measured ≈ 0.18 s and 0.20 s respectively, small and tightly clustered, ruling out network or relay transit as the source of a large visible lag.

6.5.2 Instrumentation: `monitor_goal_timing.py`

A dedicated instrumentation tool was built to timestamp a single Nav2 goal end to end: goal-sent, real motion start/stop (from `/odom`), RViz-displayed motion start/stop (from the `map` → `base` TF), and goal-reached. Two implementation defects were found and fixed while building this tool, both worth recording as general lessons for any future action-status monitoring code on ROS 2:

1. **A QoS durability mismatch.** The actual publisher QoS for `/navigate_to_pose/_action/status` is `RELIABLE + TRANSIENT_LOCAL`. `rclpy`’s default subscription QoS, constructed by passing a plain integer depth, leaves durability at `VOLATILE` – technically QoS-compatible by specification (a subscriber may request less than a publisher offers), but this delivered *zero* messages in practice, confirmed twice on real hardware. The fix required constructing an explicit `QoSProfile` matching the publisher exactly, a reminder that formal QoS compatibility rules do not guarantee the practical behaviour a developer expects (Section 2.1). This is exactly the `TRANSIENT_LOCAL`-publisher / `VOLATILE`-subscriber case Table 2.1 marks as formally compatible: durability governs whether *already-published*

samples are replayed to a newly matched subscriber, and a low-frequency topic like action status may not publish a fresh sample again for a long time, so a **VOLATILE** subscriber that missed the replay is left waiting with no error raised on either side.

2. **Goal-detection logic.** The tool originally waited for status code **ACCEPTED** specifically to mark “goal sent”, but a goal can transition directly from absent to **EXECUTING** between two consecutive status publishes, skipping an independently observable **ACCEPTED** update entirely – this topic’s **TRANSIENT_LOCAL** history appears to retain only the latest status per goal, not the full transition sequence. The fix tracks goal UUIDs instead: the first appearance of a previously unseen `goal_id` (after seeding a known-ID set from whatever the topic already retained at monitor startup) marks “goal sent”, regardless of which status value it first carries.

Listing 6.5 shows the explicit `QoSProfile` that fixed the first defect – matching `bt_navigator`’s publisher exactly rather than relying on the default subscription QoS constructed from a plain integer depth.

```

1 STATUS_QOS = QoSProfile(
2     reliability=QoSReliabilityPolicy.RELIABLE,
3     durability=QoSDurabilityPolicy.TRANSIENT_LOCAL,
4     history=QoSHistoryPolicy.KEEP_LAST,
5     depth=10,
6 )
7 ...
8 self.create_subscription(GoalStatusArray, '/navigate_to_pose/_action/status',
9                           self.status_cb, STATUS_QOS)

```

Listing 6.5: Explicit QoS profile matching `bt_navigator`’s action-status publisher (`scripts/monitor_goal_timing.py`).

Listing 6.6 shows the second fix: rather than waiting for a specific status code, the callback seeds a `known_goal_ids` set from whatever the topic already retains at startup (so pre-existing, already-completed goals are not mistaken for a new one), then marks “goal sent” the first time a previously unseen goal UUID appears in any status, independent of which status value that UUID first carries.

```

1 if not self.known_goal_ids:
2     for s in msg.status_list:
3         self.known_goal_ids.add(bytes(s.goal_info.goal_id.uuid))
4     self.get_logger().info(
5         'seeded %d pre-existing goal id(s), waiting for a new one'
6         % len(self.known_goal_ids))
7     return
8
9 for s in msg.status_list:
10     gid = bytes(s.goal_info.goal_id.uuid)
11     if gid not in self.known_goal_ids:
12         self.known_goal_ids.add(gid)
13         if self.goal_sent_t is None:

```

```

14     self.goal_sent_t = now
15     self.tracked_goal_id = gid

```

Listing 6.6: Goal-UUID-based “goal sent” detection (`scripts/monitor_goal_timing.py`).

An explicit discovery-wait (polling `get_publishers_info_by_topic` plus a settle margin) was also required before treating the monitor as ready to observe a goal – a subscription created and immediately assumed matched can miss a fast goal’s status entirely if DDS discovery with the separate `bt_navigator` process has not yet completed.

6.5.3 Measured Result

Table 6.2 presents one complete goal traced with this tool.

Table 6.2: Timing trace of one Nav2 goal (`monitor_goal_timing.py`).

Event	Offset	Pose
Goal sent	–	–
Real start (odom)	+2.145 s after goal sent	(3.344, 1.674) odom frame
RViz start (map→base)	+2.705 s after goal sent	(0.033, –0.031) map frame
Goal reached	+6.660 s after goal sent	–
RViz stop	+2.094 s after goal reached	(1.257, –0.024)
Real stop (odom)	+2.238 s after goal reached	–

This gives a start gap (RViz minus real) of +0.559 s and a stop gap of –0.144 s (RViz reporting “stopped” slightly *before* the real vehicle fully settled – small, and plausibly within the detection threshold’s own noise rather than a separate effect). Note that the real-start offset of 2.145 s already includes the static-friction stiction delay characterised in Section 5.5.2, so it is not, by itself, a new timing number.

6.5.4 Interpretation, and a Necessary Caveat

This measurement supports two separate, unrelated conclusions. The ≈ 0.56 s start-of-goal gap is a modest, constant pipeline latency – network transit, AMCL’s scan-matching cycle, and RViz’s own subscribe/render overhead stacking up – between a real event and its appearance on screen; it is not a TF-jump defect (ruled out in Section 6.5.1), and it is not a real navigation-performance problem, since Nav2’s own controller consumes `/odom//amcl_pose//tf` directly at the source, without RViz’s additional rendering overhead – the robot’s actual control decisions are working with data less stale than what a human operator sees on screen. The end-of-goal gap, by contrast, is the same stop-latency/coast phenomenon already characterised in Chapter 5: the real vehicle continuing to move for 2.238 s after “goal reached” matches the pre-fix stop-latency figures in Table 5.2’s predecessor almost exactly, confirmed here via a completely independent measurement method (a live Nav2 goal rather than the open-loop `drive_and_log.py` test).

The necessary caveat, and the reason this section is presented in detail rather than as a closed result: **this entire measurement was taken while the `readMode=1` buffering defect of Section 5.6 was still active** in `qcar_bridge.py` – it was fixed the following day. Given that defect’s confirmed effect on every encoder-derived quantity (Table 5.3), the 2.238 s end-of-goal

figure here is almost certainly, in large part, the same artefact rather than a distinct navigation-specific finding – its magnitude matches the buffering defect’s characterised effect too closely to be coincidental. The 0.559s start-gap figure is a different measurement (`map→base` TF timing rather than `/odom` velocity thresholding) and is less obviously the same artefact, but since it is also downstream of the same buffered `/odom` stream, it cannot be treated as a clean, artefact-free measurement of pure network/AMCL/RViz pipeline latency either. The diagnostic methodology and tooling built in this section – the QoS/goal-detection fixes, and the systematic elimination of TF-jump, Fixed-Frame, and network-latency causes – remain valid and reusable; only the specific numbers in Table 6.2 should be treated as provisional pending a fresh `monitor_goal_timing.py` run with the `readMode=0` fix in place.

6.6 Final Validation

Following the `readMode=0` fix (Section 5.6), a live Nav2 goal was re-run on the physical vehicle – full onboard bridge, LiDAR node, and relay node, with AMCL localised against the existing saved map. The vehicle’s operator’s direct, contemporaneous assessment was unambiguous: navigation was “working almost perfectly” – a strong qualitative result, and one consistent with the buffering defect having been the dominant cause of the “skeleton of the map is moving” symptom that opened Section 6.5, rather than merely a contributing factor to it. `monitor_goal_timing.py` was not re-run for this specific goal, so the precise post-fix start-gap and end-of-goal figures corresponding to Table 6.2 are not available as quantitative numbers in this thesis.

6.7 Summary

This chapter closes the sim-to-real migration begun in Chapter 3: the physical QCar, connected through the bridge of Chapter 4 and calibrated per Chapter 5, was shown to complete real, closed-loop Nav2 navigation goals, including an Ackermann-appropriate custom MPPI critic and phased integration that isolated each subsystem’s own correctness before combining it with the next. Section 6.4’s custom smoother resolved a genuine execution-side freeze at reversal cusps – reducing a representative K-turn manoeuvre’s simulated mean duration from 110.2s with recoveries to 43.9s with none, and validated directly on the physical vehicle across two independent goals with zero recoveries in either – while its two reverted follow-on attempts, and the still-open gap between simulated and real replan complexity, are reported alongside it rather than omitted. The one remaining significant open item is quantitative rather than functional: this chapter’s display-lag investigation (Section 6.5) was measured under a bridge configuration later found to be affected by the buffering defect of Section 5.6, and a qualitative post-fix confirmation exists, but a fresh quantitative re-measurement does not. Chapter 7 draws together the quantitative results available across this and the preceding two chapters.

7 Results

This chapter consolidates the quantitative results produced across Chapters 3–6 and evaluates them directly against the target state defined in Section 1.2: a Nav2 navigation stack that reaches goal poses reliably on the physical QCar, with the sim-to-real gap identified and, where practical, corrected. Results are presented here without re-opening their diagnosis; interpretation and generalisation are deferred to Chapter 8.

7.1 Actuator Calibration Accuracy

Table 7.1 restates the throttle calibration validation of Section 5.5.3 alongside the uncalibrated baseline it replaced.

Table 7.1: Commanded-versus-real cruise speed, before and after calibration.

Commanded speed	Baseline ratio (uncalibrated THROTTLE_GAIN = 1/3)	Calibrated ratio (Eq. 5.2)
0.3 m/s	$\approx 2.6\text{--}2.7$	1.05
0.4 m/s	– (not separately measured)	1.05
0.5 m/s	– (not separately measured)	1.00

The commanded-to-real speed ratio was reduced from $\approx 2.6\text{--}2.7\times$ to within 5% across the 0.3 m/s to 0.5 m/s range validated. Below ≈ 0.25 m/s, both the calibrated and uncalibrated mappings are unreliable in an open-loop sense, since the vehicle’s static-friction deadband (Section 5.5.2) sits in that range independently of gain calibration; commanding 0.2 m/s moved the vehicle in only some trials.

7.2 Stop-Distance and Latency

Table 7.2 restates the `readMode=0` fix’s effect (Section 5.6.4) across all five tested conditions, alongside the residual physical explanation for what remains.

Table 7.2: Stop-latency and extra-distance results, before and after the `readMode=0` fix.

Test condition	Stiction delay (before $\rightarrow$ after)	Extra distance (before $\rightarrow$ after)	Stop latency (before $\rightarrow$ after)
2 m @ 0.3 m/s	2.09 s to 0.117 s	0.520 m to 0.114 m	2.37 s to 0.598 s
2 m @ 0.4 m/s	2.14 s to 0.001 s	0.790 m to 0.114 m	2.40 s to 0.501 s
2 m @ 0.5 m/s	2.14 s to 0.116 s	0.923 m to 0.154 m	2.55 s to 0.550 s
1 m @ 0.3 m/s	2.14 s to 0.120 s	0.641 m to 0.118 m	2.44 s to 0.572 s
3 m @ 0.3 m/s	2.08 s to 0.088 s	0.583 m to 0.104 m	2.35 s to 0.509 s

Mean stiction delay across the five post-fix conditions is ≈ 0.09 s (down from ≈ 2.12 s), mean extra distance is ≈ 0.12 m (down from ≈ 0.69 m), and mean stop latency is ≈ 0.55 s (down from ≈ 2.42 s). The post-fix residual is not further reducible with the current open-loop throttle model:

dividing extra distance by stop latency recovers 54 % to 61 % of cruise velocity in four of the five conditions, consistent with an ordinary, roughly linear velocity decay to zero over the measured latency window (a perfectly linear decay predicts exactly 50%), rather than indicating any further unexplained delay mechanism.

7.3 Sim-versus-Real Overshoot

Table 7.3 restates the sim-versus-real comparison of Section 5.4. Because this comparison was run *before* both the throttle calibration (Section 5.5) and the buffering-defect fix (Section 5.6), it is reported here as the gap that existed at that stage of the project, not as the final residual gap between simulation and hardware – Sections 7.1 and 7.2 above are the corresponding final, post-fix figures, but a matched six-point sim-versus-real sweep was not re-run after both fixes were in place.

Table 7.3: Sim-versus-real stop-distance overshoot (measured before the fixes of Ch. 5).

	Simulation	Real hardware
Overshoot range (six-point sweep)	0.044 m to 0.067 m	0.58 m to 2.12 m
Isolated 2 m@0.3 m/s test	0.067 m extra	1.90 m extra
Ratio (real / sim), isolated test		$\approx 28\times$
Speed sensitivity	weak	strong ($\propto v^3$, approximately)

7.4 Steering Response Accuracy

Table 7.4 restates the steering-gain validation of Section 4.5.1, isolating the multiplicative gain error from the zero-point trim already covered by Section 7.1’s companion actuator.

Table 7.4: Real wheel deflection versus kinematically commanded deflection, before and after STEERING_GAIN correction.

	Uncorrected (gain= 1)	Corrected (STEERING_GAIN= 1.39)
Residual error vs. kinematic prediction	$\approx 39\%$	$\approx 5.6\%$

As with the throttle mapping, this is a per-unit, empirically fitted correction rather than a property derivable from the vehicle’s nominal kinematic model alone – a second, independent confirmation of this thesis’s broader finding that the QCar’s actual actuator response cannot be assumed to match its documented/kinematic specification without direct measurement.

7.5 Navigation Robustness in Tight Spaces

Table 7.5 restates Section 6.4’s K-turn/reversal-cusp results: the representative corridor reproduction goal’s manoeuvre duration and recovery count, across the untouched baseline and the final, corner-extending CuspStraightenerSmoother configuration.

This is corroborated, not merely repeated, by real-hardware testing: two independently sent goals on the physical vehicle, each requiring a genuine reversal manoeuvre, both succeeded with

Table 7.5: K-turn manoeuvre duration and recovery count, before and after the cusp-straightening smoother (simulation, three trials each).

	Baseline (no smoother)	Final config (smoother + corner extension)
Mean duration	110.2 s	43.9 s
Mean recoveries per trial	2.0	0.0
Worst-case duration	168.1 s	48.1 s

zero recoveries (Section 6.4.6) – a stronger form of validation than the simulation trial count alone, since it confirms the fix transfers to real sensor noise, real actuator response, and real planner behaviour on unseen goal geometry, not only the specific corridor reproduction it was developed against. The one qualification recorded against this result is that individual replans on real hardware occasionally showed substantially higher cusp counts (up to 8–10) than any simulation trial produced (maximum ≈ 3) – both hardware goals still succeeded despite this, so it is reported as an open, unexplained gap between simulated and real replanning behaviour rather than a limitation of the fix itself.

7.6 Navigation Goal-Reaching

Closed-loop `navigate_to_pose` goals requiring both straight driving and real turning were completed successfully on the physical QCar on multiple occasions across this project, confirmed both by `bt_navigator`’s own `Goal succeeded` status and, in the final validation of Section 6.6, by the operator’s direct observation that localisation tracked the vehicle’s real motion closely throughout the goal, rather than exhibiting the map-shift artefact of Section 6.5. The one navigation timing measurement available (Table 6.2) was taken under the `pre-readMode=0` bridge configuration and should be read as an upper bound on the true post-fix pipeline latency and stop-coast behaviour during a live goal, not as the final figure, per the caveat in Section 6.5.4.

7.7 Applicability of the Results

7.7.1 Beyond This Specific QCar Unit

The throttle deadband/gain values (Equation 5.2) and the steering-trim value (Section 4.5) are per-unit calibration constants, expected to differ, potentially substantially, on a different physical QCar chassis, or even on this same chassis under different tyre wear, floor surface, or battery charge conditions – the calibration explicitly found this floor to drift within a single session (Section 5.5.2). The *method* used to find them – an empirical fixed-duty sweep under real load, rather than an analytically derived or single-point gain estimate – is expected to transfer directly to any similarly actuated small-scale ground vehicle, independent of the specific numeric result.

7.7.2 Beyond the QCar Platform

The `readMode` buffering defect (Section 5.6) is specific to the Quanser HAL’s task-based I/O mode, but the general failure class it represents – a background acquisition task started at object-construction time, with a read-consumption rate that happens to match its production rate closely

enough that a start-up-gap backlog never drains – is a generic ring-buffer hazard that is not specific to this vendor’s library, and is worth checking for in any hardware abstraction layer offering a buffered or task-based I/O mode as an alternative to immediate/pollled reads, particularly where the buffer is initialised eagerly rather than on first read. The diagnostic principle that resolved it – when an internally self-consistent software abstraction (here, the `Odometry` class) produces a value that contradicts direct physical observation, verify the abstraction’s own *input*, not only its internal consistency – is a general debugging principle, not specific to this project at all.

7.7.3 Beyond Navigation

The active-braking safety incident (Section 5.3) and its resolution generalise directly to any reactive feedback-control design intended for direct hardware deployment on a system with confirmed actuator lag or inertia, regardless of application domain: this project’s standing rule – trace new reactive control logic through by hand for an overshoot scenario, or validate it in simulation, before it reaches a physical actuator – is a general control-engineering caution reinforced, not discovered, by this project’s own experience.

8 Summary and Prospects

8.1 Summary

This thesis set out to migrate a Nav2-based navigation stack for an Ackermann-steered vehicle, the Quanser QCar, from a validated Gazebo simulation onto the physical robot, and to identify and address the sim-to-real gap encountered along the way (Section 1.2). Each of the four sub-questions posed in the introduction is answered directly by one methodology chapter:

- **Simulation-stack structure** (Chapter 3): the stack was migrated twice – first from a hand-rolled bicycle-model inversion to `ros2_control`’s Ackermann controller (deliberately adopting real dead-reckoning odometry rather than simulation ground truth, so the eventual hardware migration would not have to absorb that discrepancy on top of everything else), then to a direct Gazebo drive plugin – which, as Section 3.4.1 documents, quietly reverted `/odom` back to ground truth despite the topic name and intent staying the same, a discrepancy this thesis surfaces rather than leaves uncorrected.
- **Bridging incompatible ROS 2 distributions** (Chapter 4): native ROS 2 communication between the vehicle’s onboard ROS 2 Dashing and the development host’s ROS 2 Humble was confirmed, through direct testing rather than assumption, to be impossible at the DDS protocol level. A TCP/JSON bridge was designed and built that presents the physical robot to the rest of the navigation stack as an ordinary ROS 2 robot, with LiDAR angle-convention, odometry, steering-trim, and inter-machine clock-synchronisation corrections layered on top to bring the bridged data to a fidelity usable by AMCL and Nav2.
- **Actuator identification and calibration** (Chapters 4–5): an uncalibrated throttle mapping ($2.6\text{--}2.7\times$ overshoot) was corrected to within 5% accuracy across 0.3 m/s to 0.5 m/s via an empirical duty sweep; a large ($\approx 2\text{ s}$) apparent coasting delay was ultimately traced not to physical drivetrain dynamics but to a ring-buffer artefact in the vendor hardware abstraction layer’s task-based I/O mode, reducing stiction delay and stop latency by more than an order of magnitude once corrected; and, independently, a real $\approx 39\%$ steering-gain error – separate from, and not fixed by, the vehicle’s zero-point trim – was characterised through a seven-point sweep and corrected to $\approx 5.6\%$ residual error (Section 4.5.1).
- **Closed-loop validation** (Chapter 6): the bridged, calibrated vehicle was shown to complete real Nav2 goals requiring both straight driving and turning, using a custom Ackermann-appropriate MPPI critic and a phased integration approach that isolated each subsystem before combining it with the next. A separate execution-side freeze at reversal cusps during K-turn manoeuvres in tight spaces was diagnosed to MPPI’s own cusp-arrival gating and resolved with a custom path-smoothing plugin, cutting a representative manoeuvre’s mean duration from 110.2 s (with recoveries) to 43.9 s (with none) in simulation, and validated directly on the physical vehicle across two independent goals with zero recoveries in either (Section 6.4).

Chapter 7 quantified the resulting gap: actuator response now tracks commanded speed within 5% (down from a $2.6\text{--}2.7\times$ error), and stop-latency/coast behaviour was reduced from roughly 2.4s and up to $3\times$ the commanded stop distance to approximately 0.5s and ≈ 0.1 m, a residual fully explained by ordinary linear velocity decay rather than any remaining unexplained mechanism.

8.1.1 Critical Reflection

What worked well in this project was the discipline of measuring directly on hardware rather than trusting a plausible-sounding explanation: four of this thesis’s most significant findings (the protocol-version incompatibility in Section 4.1, the sim-vs-real overshoot ratio in Section 5.4, the buffering defect in Section 5.6, and the reversal-cusp freeze in Section 6.4) were each, at some point, plausibly attributable to a simpler cause – localisation error, a request-delivery bug, physical coasting, a tuning-only gain problem – that direct measurement subsequently ruled out. The cusp-freeze investigation additionally illustrates the same discipline applied to solutions, not only diagnoses: three plausible fix attempts and two later follow-on refinements were each tested directly against hardware or a controlled reproduction, found wanting by measurement rather than intuition, and reverted with the evidence that ruled them out recorded rather than discarded. What could have gone better is equally clear from Section 5.3: a reactive control design was deployed to physical hardware before it had been adequately traced through or simulated, and it produced a genuine safety incident. That the incident caused no lasting harm does not make the shortcut it resulted from acceptable practice, and this thesis records it, unedited, as a documented counter-example to its own otherwise measurement-first methodology.

A second, more structural limitation is that not every quantitative result in this thesis is a genuinely final, post-fix measurement. Section 6.5.4 reports navigation timing figures measured while the buffering defect of Section 5.6 was still active, with only a qualitative (not re-instrumented) confirmation available after the fix, and Section 7.3’s sim-versus-real comparison likewise predates both major fixes in Chapter 5. These are reported as open items rather than papered over, and the exact next steps to close them – re-running `monitor_goal_timing.py` and the six-point sim-versus-real sweep under the final, fixed configuration – are specific and immediately actionable rather than vague.

8.2 Prospects

Beyond the two re-measurements identified above, three directions follow naturally from this thesis’s findings:

- **Closed-loop low-speed control.** This thesis’s calibration (Section 5.5.3) explicitly identified the sub-0.25 m/s regime as a hard limit of open-loop duty control, not a remaining tuning gap. Reliable low-speed creep – useful for close-quarters manoeuvring or fine final-approach positioning – would require closed-loop velocity feedback from the encoder rather than the fixed duty mapping used throughout this thesis.
- **Deceleration-profile tuning for coast reduction.** The stop-latency residual characterised in Section 7.2 is now small, but not zero, and scales with cruise speed. Given that ac-

tive braking is explicitly ruled out on safety grounds (Section 5.3), a lower-risk avenue – identified but not investigated within this thesis – is tuning the MPPI `GoalCritic`/deceleration profile in `nav2_params.yaml` so the controller commands a lower cruise speed during final approach to a goal, proportionally shrinking (not eliminating) the coast without introducing any new reactive control loop.

- **Generalising the diagnostic methodology.** This thesis’s most transferable contribution may be procedural rather than numeric: the recurring pattern of an internally self-consistent software abstraction masking a stale or defective input (Section 5.6), and the specific discipline of verifying a component’s own upstream input rather than only its internal consistency, is a general principle worth testing on other buffered or task-based hardware interfaces beyond the one vendor library examined here.
- **Closing the simulated-versus-real replan-complexity gap.** Section 6.4.6 found that real-hardware K-turn manoeuvres occasionally produce substantially higher per-replan cusp counts (8–10) than any simulation trial reproduced (maximum ≈ 3), without preventing either tested goal from succeeding. The leading candidate explanations – `SmacPlannerHybrid`’s sensitivity to real, more awkward goal/environment geometry, or a residual, session-specific clock-offset effect similar to the one implicated in Section 6.4.9’s reverted `IsPathValidDebounced` attempt – were identified but not isolated, and remain open. Relatedly, a deviation-aware critic weighting scheme – easing a sampled trajectory’s pull back toward the exact planned path once it has deviated significantly, in favour of a goal-direction target, rather than insisting on precise reconvergence – was proposed but not implemented, as a controller-side complement to Section 6.4’s path-preprocessing fix.
- **Hardware re-validation of the curvature-continuing smoother refinement.** Section 6.4.7’s two most recent refinements – the `min_lead_distance` fix to the cusp re-extension bug, and the curvature-continuing extension itself – were, at the time of writing, verified only in simulation (a single smoke test for the curvature refinement, two trials without directly observing the skip branch fire for `min_lead_distance`). Both are strict generalisations of the already hardware-validated straight-line mechanism and are expected to transfer, but this thesis deliberately does not claim hardware confirmation it does not have; repeating Section 6.4.6’s two-goal real-vehicle test against this refined configuration is the immediate next step.

The QCar’s navigation stack, as delivered by this thesis, reaches real closed-loop navigation goals on physical hardware with a sim-to-real gap that is now measured, explained, and in most respects substantially reduced – the original goal stated in Section 1.2. What remains open is narrower and better characterised than what this thesis began with: not “why doesn’t this work,” but two specific, already-scoped measurements and two specific, already-identified directions for further improvement.

Bibliography

- [1] eProsima. Fast dds documentation, 2024. <https://fast-dds.docs.eprosima.com/>, accessed 2026-08-23.
- [2] Wolfgang Hess, Damon Kohler, Holger Rapp, and Daniel Andor. Real-time loop closure in 2d lidar slam. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, 2016.
- [3] Nathan Koenig and Andrew Howard. Design and use paradigms for gazebo, an open-source multi-robot simulator. In *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, volume 3, pages 2149–2154, 2004.
- [4] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66), 2022.
- [5] Steven Macenski, Francisco Martín, Ruffin White, and Jonatan Gines Clavero. The marathon 2: A navigation system. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.
- [6] Yuya Maruyama, Shinpei Kato, and Takuya Azumi. Exploring the performance of ros2. In *Proceedings of the 13th International Conference on Embedded Software (EMSOFT)*, 2016.
- [7] Object Management Group. *Data Distribution Service (DDS), Version 1.4*, 2015. OMG Document formal/2015-04-10.
- [8] Quanser Inc. *QCar User Manual*, 2022. Product documentation, Quanser Inc.
- [9] Morgan Quigley, Ken Conley, Brian Gerkey, Josh Faust, Tully Foote, Jeremy Leibs, Rob Wheeler, and Andrew Y. Ng. Ros: an open-source robot operating system. In *ICRA Workshop on Open Source Software*, 2009.
- [10] Rajesh Rajamani. *Vehicle Dynamics and Control*. Springer, 2nd edition, 2011.
- [11] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. *Probabilistic Robotics*. MIT Press, 2005.
- [12] Grady Williams, Andrew Aldrich, and Evangelos A. Theodorou. Model predictive path integral control: From theory to parallel computation. *Journal of Guidance, Control, and Dynamics*, 40(2):344–357, 2017.
- [13] Grady Williams, Paul Drews, Brian Goldfain, James M. Rehg, and Evangelos A. Theodorou. Aggressive driving with model predictive path integral control. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1433–1440, 2016.

- [14] Wenshuai Zhao, Jorge Pena Queralta, and Tomi Westerlund. Sim-to-real transfer in deep reinforcement learning for robotics: a survey. In *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 737–744, 2020.

A Appendix

A.1 QCar Hardware Safety Limits

Table A.1 lists the documented hardware safety limits enforced throughout every piece of drive-control code presented in this thesis (Section 2.6).

Table A.1: QCar documented hardware safety limits.

Limit	Value
PWM duty cycle saturation	$\pm 30\%$
PWM duty cycle rate limit	100%/s
Battery auto-shutdown	below 10.0 V
Battery low-voltage warning	below 10.5 V
Steering command range	± 0.5 rad
Max. physical steering angle	± 0.5236 rad ($\pm 30^\circ$)
Overcurrent protection	5 A/8 s, 10 A/2 s, or 15 A/0.5 s $\rightarrow$ forced Neutral mode

A.2 Diagnostic and Calibration Tooling Reference

Table A.2 summarises the diagnostic scripts developed and used throughout Chapters 4 –6. Full source is maintained in the project’s version-controlled repository (electronic submission), not reproduced here in full per the guideline’s recommendation that programming code be supplied on the electronic data carrier rather than printed.

Table A.2: Diagnostic and calibration scripts referenced in this thesis.

Script	Purpose
<code>calibrate_steering.py</code>	Static steering-trim calibration aid (Section 4.5)
<code>drive_and_log.py</code>	Drive to a target distance at a fixed speed, log <code>/odom</code> to CSV, exit automatically (Section 5.5.3)
<code>check_stop_latency.py</code>	As above, plus reports cruise velocity and stop latency directly (Section 5.5.2)
<code>monitor_goal_timing.py</code>	Instruments one Nav2 goal end-to-end: goal-sent, real/displayed motion start and stop, goal-reached (Section 6.5.2)
<code>log_goal.py</code>	Logs every <code>/plan</code> replan (sequence-numbered CSVs), <code>/odom</code> , and <code>/cmd_vel</code> for one goal, for offline K-turn/cusp analysis (Section 6.4.1)
<code>CuspStraightenerSmoother</code>	<code>nav2_core::Smoother</code> plugin resolving the reversal-cusp freeze (Section 6.4.4); locked-in production configuration
<code>IsPathValidDeboundedCondition</code>	Custom BT.CPP condition node requiring consecutive failed validity checks before replanning; implemented and hardware-tested, currently reverted/deselected (Section 6.4.9)
<code>PathDeviationCritic</code>	Custom MPPI critic pulling deviated trajectories toward a lookahead path point; implemented and tested, currently reverted/deselected (Section 6.4.9)

A.3 Real-Hardware Bring-Up Command Reference

For reproducibility, the full bring-up sequence used throughout Chapters 4–6 is reproduced here.

```
1 # --- On the QCar (two terminals) ---
2 sudo -E env PYTHONPATH=/home/nvidia/Documents/python \
3     python3 ~/ros2_amit/src/onboard_bridge/qcar_bridge.py
4
5 PYTHONPATH=/home/nvidia/Documents/python \
6     python3 ~/ros2_amit/src/onboard_bridge/qcar_lidar_node.py
7
8 # --- On the development PC ---
9 ros2 run qcar_updated qcar_relay_node.py --ros-args \
10     -p qcar_ip:=<qcar-ip> -p qcar_clock_offset:=<measured-offset>
11
12 ros2 launch qcar_updated qcar_nav2.launch.py launch_sim:=false
```

Listing A.1: Real-hardware bring-up (two QCar terminals, two development-PC terminals).

`qcar_clock_offset` (Section 4.6) and the QCar’s DHCP-assigned IP address both drift session to session and must be re-measured / re-checked before each bring-up, not reused from a previous session.

A.3.1 Safe Shutdown Sequence

The shutdown sequence established in Section 4.7 must be followed in order, using `SIGINT` (Ctrl-C) exclusively:

1. Stop `qcar_relay_node.py` (development PC).
2. Stop the visualisation launch process group (development PC).
3. Stop `qcar_lidar_node.py` (QCar).
4. Stop `qcar_bridge.py` (QCar, requires `sudo`).
5. Confirm both TCP ports are released on the QCar: `sudo ss -tlnp | grep -E "5555|5556"` should print nothing.

A.4 Source Code and Data Availability

The complete, version-controlled source code for the simulation stack (Chapter 3), the hardware bridge (Chapter 4), and all calibration/diagnostic tooling (Appendix A.2) referenced throughout this thesis is supplied on the accompanying electronic data carrier / repository: TODO: path or URL.